

Regression, Perceptrons, Backpropagation and Convolutional Neural Networks

Learning the posterior $P(y | x)$ with neural networks
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

1 October, 2026

What this lecture covers

Regression versus Bayes

Linear regression, the perceptron

Multi-layer perceptrons

Convolutional neural networks

AlexNet on ImageNet

Supervision and skip connections

Summary

Colours used in the figures

The colours carry meaning, and they are the same in every figure drawn for these notes. Where a colour only tells items apart, the caption says so.

- ▶ **Red** marks the positive class $y = +1$, the output units of a network, and max pooling.
- ▶ **Blue** marks the negative class $y = -1$ and the input: the image and the input units.
- ▶ **Purple** marks the hidden units and the feature maps.
- ▶ **Teal** marks what has been learnt: a fitted line, a final hyperplane, a trained network, a correct label, and skip connections.
- ▶ **Orange** marks the backward pass: gradients, errors and losses, and also upsampling.
- ▶ **Olive** marks a model that fails on the data at hand.

From Bayes decision theory to learning the posterior

Bayes decision theory (BDT) poses a visual task with two probability distributions. The likelihood $P(x | y)$ says how an image x is generated from a state of the world y , and the prior $P(y)$ says how often each state occurs. Bayes' rule combines them into the posterior $P(y | x)$, the probability of each state once the image has been seen, and the decision is taken from the posterior.

Lecture 8 asked where the numbers in such distributions come from. Their parameters θ are learnt from training data by maximum likelihood (ML), and by expectation-maximisation (EM) when part of the data is hidden. Both learn a model of the data: a distribution that could generate it.

This lecture takes a shortcut. A deep network with weights ω outputs a distribution $P(y | x, \omega)$ over the labels directly, and the weights are learnt from pairs (x, y) of images and labels. The new question is: can we learn the posterior $P(y | x)$ directly from pairs (x, y) , without a model of the images?

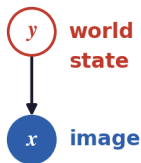
From Bayes decision theory to learning the posterior

A word on letters, because three lectures meet here. Lecture 8 wrote the parameters of a network as α . The course notes on regression (Yuille, 2024) write them ω , and these notes follow them, so that α keeps its meaning from Bayes decision theory: $\alpha(x)$ is the decision rule, the decision taken for the observation x . The observation x is usually an image, and y is a label: the name of an object, or edge and non-edge at a pixel.

The figure sets the three models side by side. In each, an arrow points in the direction in which the model computes: from the world to the image in Bayes decision theory, from the parameters to the data in Lecture 8, and from the image to the label in this lecture.

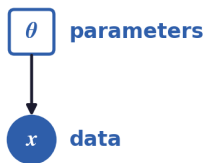
From Bayes decision theory to learning the posterior

Bayes decision theory



$P(x|y)P(y)$
gives $P(y|x)$

Lecture 8: learning θ



ML and EM
estimate θ

this lecture: regression



a network gives
 $P(y|x, \omega)$

Figure 1: Three models. First panel: the world state y (red circle) generates the image x (blue disc), and $P(x|y)P(y)$ gives the posterior $P(y|x)$ by Bayes' rule. Second panel: the parameters θ (blue square) generate the data x , and ML or EM estimates θ . Third panel: an image x enters a network with weights ω (teal box), and a label y comes out.

The decision rule depends only on the posterior $P(y \mid x)$

The Bayesian approach is formulated in terms of the joint probability distribution of the observation and the state, $P(x, y) = P(x \mid y) P(y)$. A decision rule $\alpha(x)$ assigns a decision to every observation x , and a loss function $L(\alpha(x), y)$ says how much the decision $\alpha(x)$ costs when the true state is y . With the 0-1 loss, for example, a correct decision costs 0 and every error costs 1.

The risk of a rule is its expected loss, averaged over all observations and all states. Writing the joint distribution as $P(x, y) = P(x) P(y \mid x)$ gives

$$R(\alpha) = \sum_x \sum_y L(\alpha(x), y) P(x, y) = \sum_x P(x) \sum_y L(\alpha(x), y) P(y \mid x).$$

The inner sum is the expected loss of the decision $\alpha(x)$ for one observation x , averaged over the states that could have produced it.

The risk is minimised one observation at a time

The rule chooses a decision separately for every x , and every term of the outer sum carries a weight $P(x) \geq 0$. The risk is therefore smallest when each inner sum is as small as it can be, whatever the weights $P(x)$ are:

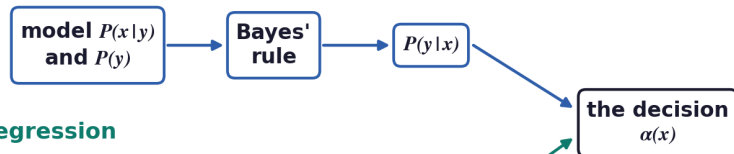
$$\hat{\alpha}(x) = \arg \min_{\alpha(x)} \sum_y L(\alpha(x), y) P(y | x).$$

This is the Bayes decision rule, and its risk, the smallest achievable, is the Bayes risk. The prior and the likelihood enter the rule only through the posterior. With the 0-1 loss the inner sum is $1 - P(\alpha(x) | x)$, and the rule picks the most probable state, $\hat{\alpha}(x) = \arg \max_y P(y | x)$: the maximum a posteriori (MAP) estimate.

So the decision needs only $P(y | x)$. The joint distribution $P(x, y)$ holds much more: how often each image occurs, $P(x) = \sum_y P(x | y)P(y)$, which the decision never uses. This suggests a question: maybe we only need to learn $P(y | x)$ instead of $P(x | y)P(y)$?

The decision rule depends only on the posterior $P(y | x)$

Bayes



regression



Figure 2: Two routes to the same decision. Upper row, in blue: the Bayesian route models how images are generated, $P(x|y)$ and $P(y)$, and applies Bayes' rule to obtain the posterior. Lower row, in teal: regression learns the weights ω of a family of distributions from training pairs (x_i, y_i) and obtains an estimate $P(y|x, \omega)$ of the posterior. Both deliver a posterior to the decision rule $\alpha(x)$.

Learning $P(y | x)$ from data is called regression

Learning a distribution $P(y | x)$ from data is called regression. It requires a family of distributions $P(y | x, \omega)$, specified by parameters ω , and a way to estimate ω from the training data

$$\mathcal{X} = \{(x_i, y_i) : i = 1, \dots, N\}.$$

The pairs are assumed to be independent samples. Maximum likelihood chooses the ω under which the observed labels are most probable given their images:

$$\hat{\omega} = \arg \max_{\omega} \prod_{i=1}^N P(y_i | x_i, \omega) = \arg \min_{\omega} \sum_{i=1}^N \{-\log P(y_i | x_i, \omega)\}.$$

The product becomes a sum of logarithms, and the maximisation a minimisation of minus that sum. The images are taken as given, so the criterion needs a model of $P(y | x)$ alone, never a model of how the images arise. For this reason modelling $P(y | x)$ directly is called a discriminative approach, and modelling $P(x | y)P(y)$ a generative one.

Learning $P(y \mid x)$ from data is called regression

Deep networks are models of this type: ω are the weights of the network, and the last layer outputs one number s_k for every label k , turned into a probability distribution by the softmax function $\exp(s_k) / \sum_j \exp(s_j)$. Confusingly, some neural network authors use “regression” only if y is a continuous variable, and call the case of discrete labels classification. In statistics any model of $P(y \mid x)$ is a regression.

The figure shows the output of one such network. AlexNet (Krizhevsky et al., 2012), a network trained on the 1,000 classes of the ImageNet dataset, gives the photograph of a cat the probabilities 0.47 for Egyptian cat, 0.34 for tiger cat, 0.17 for tabby and 0.02 for lynx. Several cat classes of ImageNet fit the photograph, and the network spreads its belief over them. The course notes ask: are current deep nets adequate to represent the true posterior $P(y \mid x)$ for vision? (The computed figures of these notes use the public BVLC re-implementation of AlexNet, trained on the same data in the same way as the original.)

Learning $P(y | x)$ from data is called regression

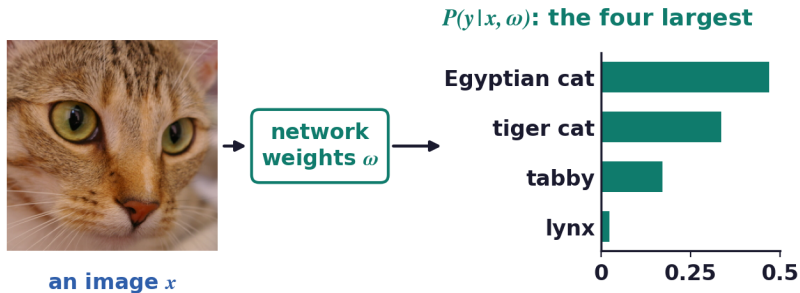


Figure 3: An image x enters a network with weights ω , and the network outputs a probability for each of the 1,000 classes of ImageNet. The teal bars are the four largest, computed with AlexNet on the photograph “chelsea” of the scikit-image sample data (public domain); the photograph is cropped to its central square and resized to 224×224 pixels.

The true posterior depends on the likelihood and the prior

From a Bayesian perspective regression is problematic. The true posterior depends on the likelihood function and the prior by Bayes' rule,

$$\underbrace{P(y|x)}_{\text{the posterior}} = \frac{\overbrace{P(x|y)}^{\text{the likelihood}} \overbrace{P(y)}^{\text{the prior}}}{P(x)}, \quad P(x) = \sum_y P(x|y) P(y).$$

A network that learns $P(y|x, \omega)$ directly has to represent, inside one function, everything that the likelihood and the prior would say.

The likelihood $P(x|y)$ is very complex: it requires specifying a probability distribution on images $\{x\}$. A colour image of 224×224 pixels is a list of $224 \times 224 \times 3 = 150,528$ numbers, and the likelihood must give a probability to every such list. Typically y is much lower dimensional than x : it is binary for edge detection, and takes 1,000 values for object classification on ImageNet.

The true posterior depends on the likelihood and the prior

For object detection or classification, where y is the name of an object, $P(x | y)$ should be capable of generating all possible images of the object. The figure shows a few of them for one animal. The same photograph of a cat is moved, enlarged, relit, shadowed and partly hidden; each version is a different image x with the same label $y = \text{cat}$, and $P(x | y = \text{cat})$ must assign probability to all of them, and to every other cat, in every pose, against every background. Images are complicated and very high dimensional, so such generative models are very difficult to learn, and the course notes say plainly that we do not yet have good enough probability models to do this.

The course notes then turn the argument round. Deep network regression models are surely not able to capture the true distribution $P(y | x)$ either. When does this matter?

The true posterior depends on the likelihood and the prior



Figure 4: Five changes that keep the label cat: position (the right-hand square), scale (the central 60% enlarged), lighting (intensities raised to the power 2.3, scaled by 0.9), a shadow (brightness falling from left to right) and an occluding patch.

Some visual tasks may not need a model of the images

The deep networks may be sufficiently good approximations. The Bayes risk is the smallest risk that any rule can achieve. A rule attains it exactly when its decisions agree with those taken from the true posterior, and an approximate posterior can do that. We may not need to achieve Bayes risk for some visual tasks and on some data domains: a rule with a slightly larger risk can be good enough.

The course notes give an example outside vision. The task is to classify surnames as Polish or British. It is sufficient to use statistics of the frequencies of letters like r, z, y and k, and there is no need for distributions $P(x | y)$ that can generate surnames. A classifier that counts four letters has a handful of parameters, while a model that generates plausible Polish surnames would have to know their spelling rules, their endings and their frequencies.

Some visual tasks may not need a model of the images

Some visual tasks may likewise not require detailed knowledge of the images. Object classification may not require knowing the details of the object. The task is to classify an object as a car, and a posterior over the labels car and not car can ignore what type of car it is, where its parts and boundaries are, and whether it is clean or dirty. A generative model of car images would have to produce all of these.

But this also depends on the data domains: the training and test sets. A data domain is the population of images from which the examples are drawn, such as photographs from the web, frames from a car's camera, or medical scans. Letter counts separate the surnames of a telephone directory; the same counts may fail on a list in which Polish names have been anglicised.

Some visual tasks may not need a model of the images

Polish Kowalczyk Wiśniewski Szymański
Grzybowski Zieliński

British Smith Jones Taylor Brown Roberts

letters r, z, y, k in orange

Figure 5: Common surnames of each country, with the letters of the example, r, z, y and k, in orange. The Polish names are full of them and the British ones have fewer of them, and no z or k, so counts of four letters discriminate without a model that generates surnames. The names are examples; no frequencies were computed for the figure.

Training and test sets are too small

The ways we evaluate the performance of artificial intelligence (AI) algorithms are problematic, because they depend on datasets that are biased and do not represent the complexity of the real world. Regression methods are trained on a dataset $\mathcal{X} = \{(x_i, y_i)\}$, and this dataset specifies a domain. Regression methods perform well on data from the same domain. The visual tasks that can be addressed are also limited, because datasets can be annotated for only a limited range of tasks: a person has to supply every label y_i .

The fundamental problem, in the words of the course notes, is that datasets are not nearly big enough and too few visual tasks are being addressed. Much tougher testing is required. Out-of-distribution testing uses test images drawn from a different distribution than the training images, and adversarial testing uses images changed on purpose, often by amounts too small for a person to notice, to make the algorithm fail.

Training and test sets are too small

High-tech companies are aware of this problem. They have gigantic datasets, much bigger than those of the academic community, and still one chief executive is quoted in the course notes: “Existing methods do not work on big datasets because there are too many corner cases.” A corner case is a rare situation that the training data barely cover.

Two failures are common. Algorithms trained on one data domain can perform badly when transferred to a different domain. Algorithms can also perform badly because of small changes in the images which would not fool a human observer. The figure tests AlexNet on six versions of the photograph of the cat: the original, a greyscale copy, the image rotated by 90 degrees, a blurred copy (a Gaussian of standard deviation 4 pixels), a noisy copy (Gaussian noise of standard deviation 40 grey levels) and a line drawing (the edges found by the Canny edge detector). The blur and the noise are applied to the 300×300 square of the photograph before it is resized to 224×224 .

Training and test sets are too small

AlexNet names the cat correctly in the first three versions, as Egyptian cat with probabilities 0.47, 0.55 and 0.58. For the other three its most probable labels are puffer (0.04), mask (0.08) and envelope (0.32). A person names the cat in all six. The blurred and the noisy images are changes of a kind that ImageNet photographs rarely contain, and the line drawing belongs to a different domain altogether: drawings instead of photographs.

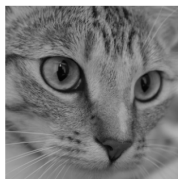
Pragmatically, it makes sense to develop deep networks and to test them on increasingly challenging datasets and visual tasks: out-of-distribution and adversarial. At worst, the course notes conclude, they serve as fast approximate algorithms, which may be sufficient for some visual tasks on some data domains.

Training and test sets are too small



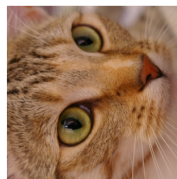
original

Egyptian cat



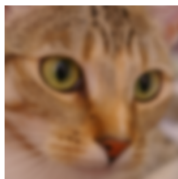
greyscale

Egyptian cat



rotated

Egyptian cat



blurred

puffer



noise

mask



line drawing

envelope

Figure 6: Six versions of the photograph and, in colour under each, AlexNet's most probable label. The grey word names the version. A teal label is a cat class of ImageNet; an olive label is another class.

Linear regression: maximum likelihood gives least squares

Linear regression is the oldest regression model. It was invented by Gauss, roughly in 1800, when he tried to find the planetoid Ceres by predicting its position from previous estimates. Piazzi had discovered Ceres on 1 January 1801 and lost it a few weeks later when it passed behind the glare of the Sun; from his few observations Gauss predicted where it would reappear, and it was found there at the end of the year. Gauss published the method of least squares in 1809 (Gauss, 1809); Legendre had published it in 1805 (Legendre, 1805).

The model expresses an output y as a linear function of an input x plus noise, $y = ax + b + \epsilon$, where a and b are unknown parameters and ϵ is zero-mean Gaussian noise of variance σ^2 . As a probability model,

$$P(y | x, a, b, \sigma) = \frac{1}{\sqrt{2\pi} \sigma} \exp \left\{ -\frac{(y - ax - b)^2}{2\sigma^2} \right\}.$$

This is a regression: a family of distributions of the output y given the input x , with parameters (a, b, σ) in the place of ω .

Maximum likelihood for the line gives two linear equations

Given training data $\{(x_n, y_n) : n = 1, \dots, N\}$, maximum likelihood estimates the parameters by

$$(a^*, b^*, \sigma^*) = \arg \max_{a, b, \sigma} \prod_{n=1}^N P(y_n | x_n, a, b, \sigma).$$

Minus the logarithm of the product is

$$N \log(\sqrt{2\pi} \sigma) + \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - ax_n - b)^2.$$

Only the sum depends on a and b , so for them maximum likelihood is least squares,

$$(a^*, b^*) = \arg \min_{a, b} \sum_{n=1}^N (y_n - ax_n - b)^2,$$

whatever the value of σ .

Maximum likelihood for the line gives two linear equations

The model is linear in a and b , hence the name linear regression, and setting the derivatives in a and b to zero gives two linear equations:

$$a \sum_n x_n^2 + b \sum_n x_n = \sum_n x_n y_n, \quad a \sum_n x_n + bN = \sum_n y_n.$$

Writing $\bar{x}$ and $\bar{y}$ for the averages of the x_n and the y_n , the solution is

$$a^* = \frac{\sum_n (x_n - \bar{x})(y_n - \bar{y})}{\sum_n (x_n - \bar{x})^2}, \quad b^* = \bar{y} - a^* \bar{x}.$$

There is also a closed form for σ^* . Setting the derivative in σ to zero gives

$$(\sigma^*)^2 = \frac{1}{N} \sum_{n=1}^N (y_n - a^* x_n - b^*)^2,$$

the mean squared residual, where a residual is the vertical distance of a point from the fitted line.

Linear regression: maximum likelihood gives least squares

In the example of the figure, 24 points are drawn from $y = 0.6x + 1.5 + \epsilon$ with $\sigma = 1$. Least squares gives $a^* = 0.584$, $b^* = 1.613$ and $\sigma^* = 1.016$, close to the values that generated the data.

The fitted model is a whole distribution of y at every x , and the figure draws three of them. At $x = 2, 5$ and 8 a purple curve is laid on its side: the Gaussian $P(y | x)$ with mean $a^*x + b^*$ and standard deviation σ^* , whose height is measured along the horizontal direction from the dotted vertical line. For a Gaussian about two points in three lie within one standard deviation of the mean; here 19 of the 24 points lie within σ^* of the line.

The method extends in many ways. With vector-valued x and y the model is $y = Ax + b + \epsilon$ and the equations for A and b are still linear. With y a non-linear parametric function of x , maximum likelihood is still least squares, but for some non-linear functions the minimisation is no longer a set of linear equations and estimating the parameters can be difficult.

Linear regression: maximum likelihood gives least squares

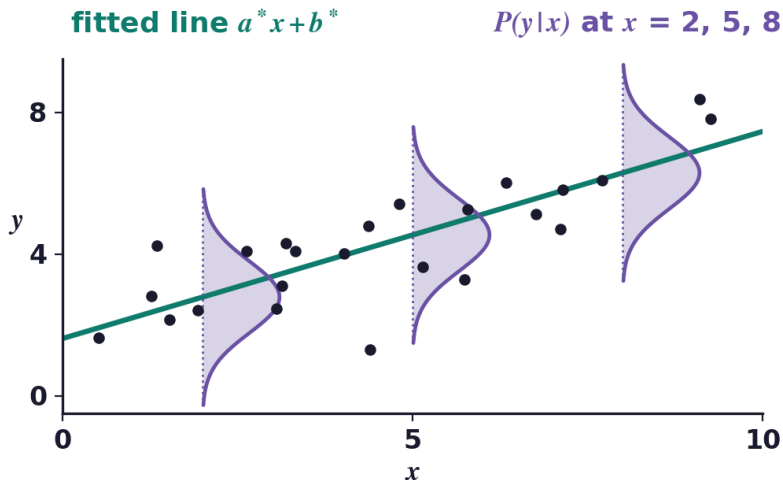


Figure 7: Linear regression on 24 points (black dots): the least-squares line $a^*x + b^*$ (teal) and the fitted Gaussian $P(y|x)$ at $x = 2, 5$ and 8 (purple curves, drawn sideways).

The perceptron classifies by the side of a hyperplane

The perceptron was developed by Rosenblatt in the 1950s and started the first wave of neural networks (Rosenblatt, 1958). It is a classifier for two classes, $y \in \{+1, -1\}$, with an input vector $\vec{x}$ and a classification rule

$$y^*(\vec{x}) = \arg \max_{y \in \{\pm 1\}} y (\vec{w} \cdot \vec{x} + w_0).$$

The rule chooses $y = 1$ if $\vec{w} \cdot \vec{x} + w_0 > 0$ and $y = -1$ if $\vec{w} \cdot \vec{x} + w_0 < 0$. The vector $\vec{w}$ and the number w_0 are the parameters.

The perceptron was based on an extremely simplified model of a neuron. The components of $\vec{x}$ are the inputs at different synapses, the contacts through which other neurons excite the cell, and $\vec{w}$ are the synaptic weights. The number $-w_0$ is a threshold: the neuron fires an action potential, a brief electrical pulse, if the total stimulation of the cell, given by the weighted sum $\vec{w} \cdot \vec{x}$ of the inputs, is bigger than the threshold.

The perceptron classifies by the side of a hyperplane

The figure is the drawing of the perceptron used in the lecture. The input x enters from below and is multiplied by the weight w . The dashed line brings in the bias b , which is the w_0 of the formula, and the unit adds the two: the label $x \cdot w + b$ is the total input. The circle is the unit itself, and the red step drawn inside it is the threshold function, which turns a negative total input into -1 and a positive one into $+1$. The output y leaves at the top, the binary classification.

With a single input x the drawing is a threshold on a number. With many inputs, x becomes the vector $\vec{x}$ and $x \cdot w$ the dot product $\vec{w} \cdot \vec{x}$.

The perceptron classifies by the side of a hyperplane

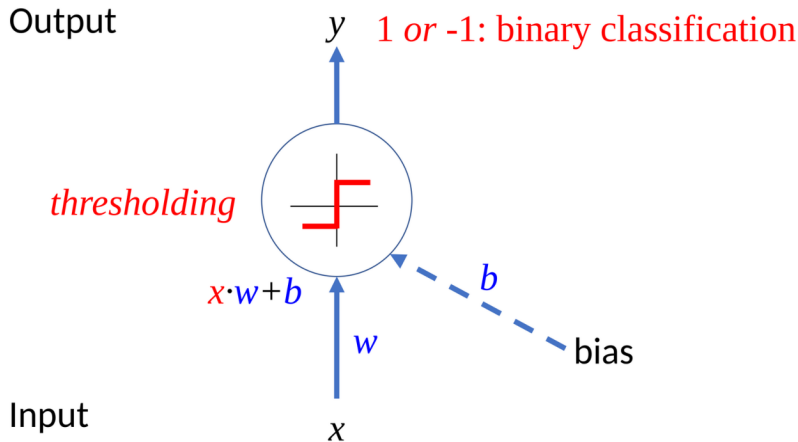


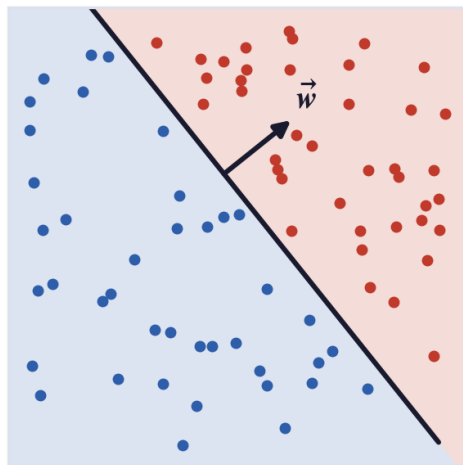
Figure 8: The threshold perceptron (from the lecture's animation of neural networks). The input x is weighted by w , the bias b is added, and the thresholding unit outputs 1 or -1.

The weights are the normal of the hyperplane

Geometrically, $\vec{w} \cdot \vec{x} + w_0 = 0$ specifies a hyperplane: a line in two dimensions, a plane in three, and a flat surface of one dimension less than the input in general. The vector $\vec{w}$ is perpendicular to it, its normal: for any two points $\vec{x}_1$ and $\vec{x}_2$ of the hyperplane, $\vec{w} \cdot (\vec{x}_1 - \vec{x}_2) = 0$.

Points $\vec{x}$ above the hyperplane, on the side to which $\vec{w}$ points, have $\vec{w} \cdot \vec{x} + w_0 > 0$ and are classified as positive, $y = 1$. Points below the hyperplane are classified as negative, $y = -1$. The number $(\vec{w} \cdot \vec{x} + w_0)/|\vec{w}|$ is the signed distance of $\vec{x}$ from the hyperplane, so the value $\vec{w} \cdot \vec{x} + w_0$ says how far from the boundary a point lies as well as on which side. Multiplying $\vec{w}$ and w_0 by the same positive number moves nothing: the hyperplane and the rule stay the same.

The weights are the normal of the hyperplane



$y = +1$: **above**

$\vec{w} \cdot \vec{x} + w_0 = 0$
the hyperplane

$y = -1$: **below**

Figure 9: Forty points of each class that one hyperplane separates. Red dots are $y = +1$ and blue dots $y = -1$. The black line is the hyperplane $\vec{w} \cdot \vec{x} + w_0 = 0$ and the arrow is its normal $\vec{w}$. The shading marks the half-plane that the rule assigns to each class.

The perceptron algorithm converges on separable data

The perceptron can be learnt from a training set $\{(\vec{x}_n, y_n) : n = 1, \dots, N\}$. Rosenblatt specified the perceptron algorithm. The examples are visited one at a time, in random order, starting from $\vec{w} = 0$ and $w_0 = 0$. Whenever an example is misclassified, or lies on the boundary, the weights are corrected towards it:

$$\text{if } y_n(\vec{w} \cdot \vec{x}_n + w_0) \leq 0 : \quad \vec{w} \mapsto \vec{w} + y_n \vec{x}_n, \quad w_0 \mapsto w_0 + y_n.$$

One correction raises $y_n(\vec{w} \cdot \vec{x}_n + w_0)$ by $|\vec{x}_n|^2 + 1$, so it moves the hyperplane in the direction that brings that example closer to its correct side. It can move other examples to the wrong side, which is why the algorithm has to keep cycling through the data.

The algorithm is guaranteed to converge to a hyperplane that separates the positive from the negative examples, provided a separating hyperplane exists. Data for which one exists are called linearly separable.

A margin bounds the number of mistakes

The guarantee has a short proof (Novikoff, 1962). Append a 1 to every input, $\tilde{\vec{x}}_n = (\vec{x}_n, 1)$, so that the weights are one vector $\tilde{\vec{w}} = (\vec{w}, w_0)$ and a correction is $\tilde{\vec{w}} \mapsto \tilde{\vec{w}} + y_n \tilde{\vec{x}}_n$. Suppose a unit vector $\vec{u}$ separates the data with margin $\gamma > 0$, $y_n \vec{u} \cdot \tilde{\vec{x}}_n \geq \gamma$ for every n , and let $R = \max_n |\tilde{\vec{x}}_n|$.

Each correction raises $\vec{u} \cdot \tilde{\vec{w}}$ by $y_n \vec{u} \cdot \tilde{\vec{x}}_n \geq \gamma$. It raises $|\tilde{\vec{w}}|^2$ by $2y_n \tilde{\vec{w}} \cdot \tilde{\vec{x}}_n + |\tilde{\vec{x}}_n|^2 \leq R^2$, because a correction happens only when $y_n \tilde{\vec{w}} \cdot \tilde{\vec{x}}_n \leq 0$. After k corrections, starting from zero,

$$k\gamma \leq \vec{u} \cdot \tilde{\vec{w}} \leq |\tilde{\vec{w}}| \leq \sqrt{k} R, \quad \text{so} \quad k \leq \frac{R^2}{\gamma^2}.$$

The middle inequality is the Cauchy-Schwarz inequality, since $|\vec{u}| = 1$. The number of corrections is finite, and when they stop every example is on its correct side. A wide margin, well-separated classes, means few corrections.

The perceptron algorithm converges on separable data

If the positive and negative examples cannot be separated by a hyperplane, the algorithm does not converge: some example is always misclassified, and the weights keep changing. Later researchers showed that in this case the average of the weights and the threshold over all the steps gives a reasonable result (Freund and Schapire, 1999): the averaged perceptron usually gives a good separator.

In the figure the separable data need 10 corrections, with 9 mistakes in the first pass through the data, 1 in the second and none after that. On the overlapping data the last eight hyperplanes classify between 69% and 92% of the points correctly, and the averaged perceptron, whose weights are the average of the weights over all the steps, 92%.

The perceptron algorithm converges on separable data

The limitation of the perceptron is that a separating hyperplane is only useful for a limited class of problems. To make it useful one would have to find, in a principled way, features $\varphi(\vec{x})$ and apply the perceptron to those. For points in a disc surrounded by a ring the feature $\varphi = |\vec{x}|^2$ makes the classes separable: all 11 corrections occur in the first pass, and the second pass finds no mistakes. Choosing such features by hand is the hard part.

The perceptron algorithm converges on separable data

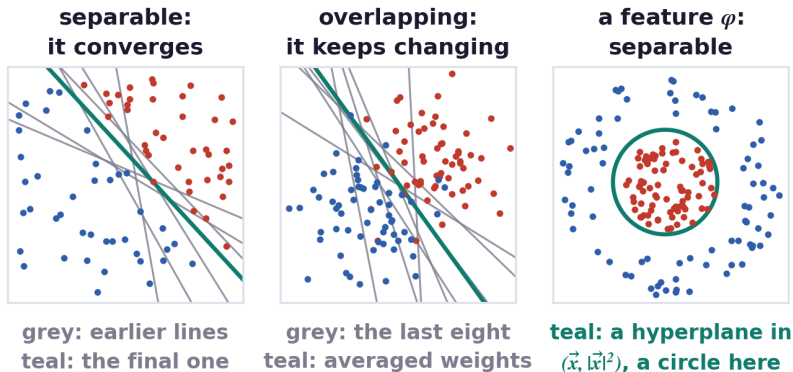


Figure 10: The perceptron algorithm on three datasets, red dots $y = +1$ and blue dots $y = -1$. Separable data: grey lines are earlier hyperplanes and teal the final one, which separates the classes. Overlapping data: grey lines are the last eight hyperplanes, which keep changing, and teal the averaged perceptron. Disc and ring: the perceptron applied to $(\vec{x}, |\vec{x}|^2)$; its hyperplane in that space is the teal circle in the plane.

The perceptron as logistic regression

The problem can be reformulated as regression, by specifying a conditional probability distribution for the label,

$$P(y | \vec{x}) = \frac{\exp\{y(\vec{w} \cdot \vec{x} + w_0)\}}{\exp\{\vec{w} \cdot \vec{x} + w_0\} + \exp\{-\vec{w} \cdot \vec{x} - w_0\}}, \quad y \in \{\pm 1\}.$$

The denominator is the sum of the numerator over the two values of y , so the two probabilities add up to one. This model is called logistic regression. Its parameters can be estimated from the training set by maximum likelihood,

$$(\vec{w}^*, w_0^*) = \arg \min - \sum_{n=1}^N \log P(y_n | \vec{x}_n),$$

and the minimisation can be performed by gradient descent, which repeatedly moves the parameters a small step against the gradient of the function.

The perceptron as logistic regression

The original perceptron assigned output values ± 1 , but these can be changed to $\{0, 1\}$ by a simple transformation, $t = (y + 1)/2$, and logistic regression is usually written with t .

The figure is the lecture's drawing of the soft perceptron. The inputs $x_1, x_2, \dots, x_n$ arrive through the weights $w_1, w_2, \dots, w_n$; the three dots stand for the inputs that are not drawn. The dashed line brings in the bias b . The red curve inside the unit is the sigmoid, an S-shaped function that rises smoothly from 0 to 1, in the place of the step of the threshold perceptron. The output y leaves at the top, and the arrows above it show that it can be the input of further units.

The perceptron as logistic regression

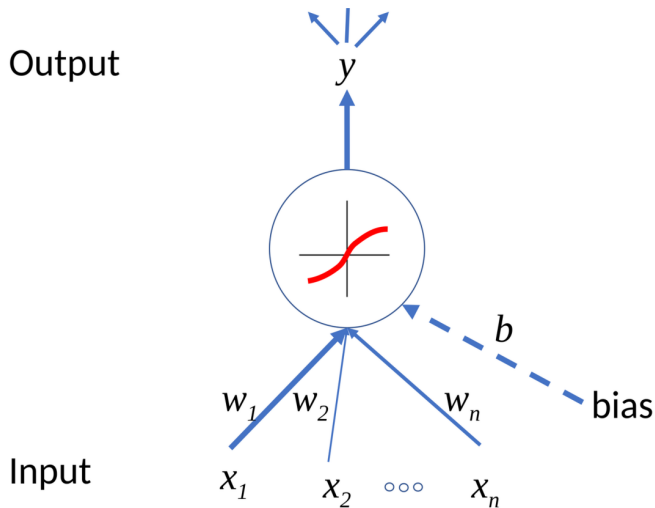


Figure 11: The soft perceptron (from the lecture's animation of neural networks): inputs x_1 to x_n with weights w_1 to w_n , a bias b , and a sigmoid unit.

Logistic regression is a sigmoid of the linear score

Write $a = \vec{w} \cdot \vec{x} + w_0$ for the linear score of an input. Dividing the numerator and the denominator of the model by e^a gives

$$P(y = 1 | \vec{x}) = \frac{e^a}{e^a + e^{-a}} = \frac{1}{1 + e^{-2a}} = \sigma(2a), \quad \sigma(u) = \frac{1}{1 + e^{-u}},$$

and $P(y = -1 | \vec{x}) = \sigma(-2a) = 1 - P(y = 1 | \vec{x})$. The function σ is the sigmoid of the drawing; this σ is a function, different from the noise level σ of linear regression.

After learning, points above the hyperplane $\vec{w} \cdot \vec{x} + w_0 = 0$ have $a > 0$ and $P(y = 1 | \vec{x}) > 1/2$, while points below it have $P(y = -1 | \vec{x}) > 1/2$. The hyperplane still separates the positive and negative examples, but the separation is soft: it is given by a number $P(y = 1 | \vec{x}) \in [0, 1]$, which tends to 1 for points infinitely far above the hyperplane and to 0 for points infinitely far below. Near the hyperplane the model is uncertain.

Fitted to the overlapping data of the figure, in the form $\sigma(\vec{w} \cdot \vec{x} + w_0)$ where the factor 2 is absorbed into the weights, the parameters are $\vec{w} = (2.768, 1.813)$ and $w_0 = -0.055$.

Logistic regression is a sigmoid of the linear score

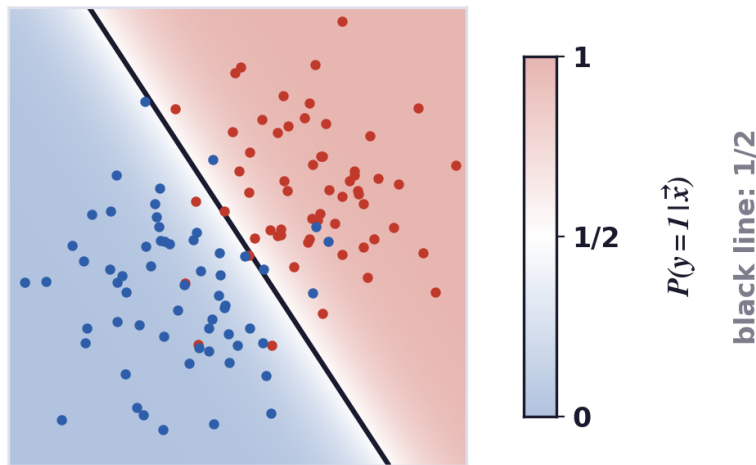


Figure 12: Logistic regression fitted to two overlapping clouds, red dots $y = +1$ and blue dots $y = -1$. The colour of the background is $P(y = 1 | \vec{x})$, from blue for 0 through white for 1/2 to red for 1. The black line, where the probability is 1/2, is the hyperplane.

Logistic regression has a convex energy: no local minima

The quantity that learning minimises is called the energy,

$$E(\vec{w}, w_0) = - \sum_{n=1}^N \log P(y_n | \vec{x}_n),$$

minus the log-likelihood of the training labels. Gradient descent updates the parameters by

$$(\vec{w}^{t+1}, w_0^{t+1}) = (\vec{w}^t, w_0^t) - \zeta_t \left(\frac{\partial E}{\partial \vec{w}}, \frac{\partial E}{\partial w_0} \right),$$

where t counts the iterations and $\zeta_t > 0$ is the learning rate, the size of the step. The learning rate has to be chosen: too large a step overshoots the minimum, too small a step takes many iterations.

The energy of logistic regression is a convex function of $\vec{w}$ and w_0 . A convex function lies below every chord, the straight segment between two points of its graph; every local minimum is a global minimum, and so is every point where the gradient vanishes.

The gradient of the logistic energy is error times input

One term of the energy, written with the score $a = \vec{w} \cdot \vec{x} + w_0$, is

$$-\log P(y \mid \vec{x}) = -ya + \log(e^a + e^{-a}).$$

Its derivative in a is $-(y - \tanh a)$, and

$\tanh a = P(y = 1 \mid \vec{x}) - P(y = -1 \mid \vec{x})$ is the expected label under the model. Since $\partial a / \partial \vec{w} = \vec{x}$ and $\partial a / \partial w_0 = 1$,

$$\frac{\partial E}{\partial \vec{w}} = - \sum_{n=1}^N (y_n - \tanh a_n) \vec{x}_n, \quad \frac{\partial E}{\partial w_0} = - \sum_{n=1}^N (y_n - \tanh a_n).$$

Each example pushes the weights by its error, the observed label minus the predicted one, times its input: examples that the model already predicts with confidence push little.

The second derivative in a is $1 - \tanh^2 a > 0$, so each term is a convex function of a . The score a is a linear function of $(\vec{w}, w_0)$, a convex function of a linear function is convex, and a sum of convex functions is convex. The energy is therefore convex.

Logistic regression has a convex energy: no local minima

Because E is convex, gradient descent with a well-chosen learning rate converges to the global optimum. The guarantee needs a finite optimum, and one exists when the two classes overlap. For separable data every term of the energy tends to zero as $\vec{w}$ grows along a separating direction, so the weights grow without bound and the minimum is never reached. A prior on $\vec{w}$, as in the maximum a posteriori estimates of Lecture 8, keeps the optimum finite. There is also a discrete update rule, the concave-convex procedure (CCCP) of Lecture 7, which converges without a learning rate.

The figure shows the energy of the overlapping data as a function of the two components w_1, w_2 of $\vec{w}$, with w_0 held at its optimal value; a slice of a convex function is convex, so the contours are those of a single bowl. Gradient descent from $(-2.2, 5)$ lowers the energy from 159.6 to 28.3 and it never rises on the way.

Logistic regression has a convex energy: no local minima

the energy with w_0 fixed: a single bowl

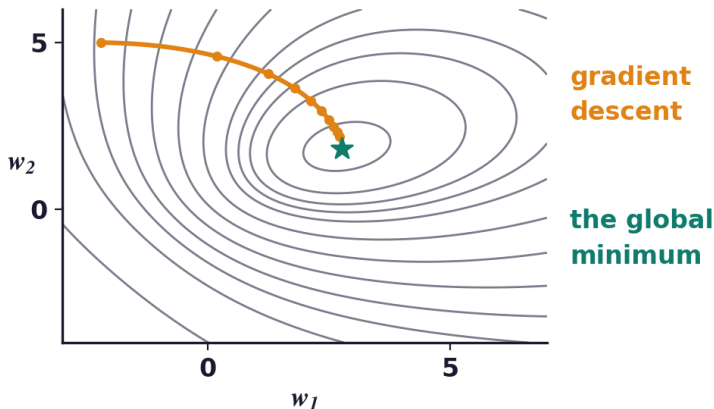


Figure 13: The energy $E(w_1, w_2)$ of logistic regression for the overlapping clouds, with w_0 fixed at its optimum. Grey curves are contours, lines of equal energy. The orange path is gradient descent, with a dot every 40 steps over the first 400, and the teal star is the global minimum.

Multi-layer perceptrons stack soft perceptrons

A soft perceptron represents its output by a continuous function, $P(y | \vec{x})$ or $\sigma(\vec{w} \cdot \vec{x} + w_0)$, which is differentiable as a function of $\vec{w}$ and w_0 . This makes it possible to build a network, called a multi-layer perceptron (MLP), by stacking perceptrons on top of each other, so that the input of a perceptron is the output of other perceptrons. The differentiability of the individual perceptrons ensures that the final output is a differentiable function of the parameters of all the neurons, which gives a natural learning rule: gradient descent on the energy, also called steepest descent.

Multi-layer perceptrons stack soft perceptrons

A simple example has two layers of units:

$$y = \sigma(\vec{w} \cdot \vec{z} + w_0), \quad z_i = \sigma(\vec{\Omega}^i \cdot \vec{x} + \Omega_0^i), \quad i = 1, 2, 3, \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

Each z_i is a soft perceptron with weights $\vec{\Omega}^i$ and bias Ω_0^i , and the output unit is a soft perceptron whose inputs are $\vec{z} = (z_1, z_2, z_3)$. The $\{z_i\}$ are called hidden variables, since they cannot be observed directly: the training data give the input $\vec{x}$ and the output y only. Here σ is the sigmoid, a different σ from the noise level of linear regression, and y is the output of the network, a number between 0 and 1; where it has to be told apart from the label it is written y_{net} .

Multi-layer perceptrons stack soft perceptrons

The weights can be learnt from training data $\{(\vec{x}_n, y_n) : n = 1, \dots, N\}$ by minimising an energy function,

$$E(\vec{w}, \{\vec{\Omega}^i\}) = \sum_{n=1}^N E(\vec{x}_n, y_n : \vec{w}, \{\vec{\Omega}^i\}),$$

where each term measures how different the true output y_n is from the predicted output of the network for $\vec{x}_n$. From the regression perspective this is maximum likelihood for

$$P(y | \vec{x}) = \frac{1}{Z} \exp\{-E(\vec{x}, y : \vec{w}, \{\vec{\Omega}^i\})\},$$

where $Z = \int dy \exp\{-E(\vec{x}, y : \vec{w}, \{\vec{\Omega}^i\})\}$ normalises the distribution.

The negative log-likelihood of an example is then $E + \log Z$, so minimising E alone is maximum likelihood only when Z does not depend on the weights.

Multi-layer perceptrons stack soft perceptrons

Two common energies satisfy the condition. For a continuous output, the squared error $(y - y_{\text{net}})^2/(2s^2)$, with a fixed s , makes $P(y | \vec{x})$ a Gaussian with $Z = \sqrt{2\pi} s$. For a binary label $t \in \{0, 1\}$ and an output y_{net} between 0 and 1, the cross-entropy $-t \log y_{\text{net}} - (1 - t) \log(1 - y_{\text{net}})$ makes $\exp\{-E\} = y_{\text{net}}^t (1 - y_{\text{net}})^{1-t}$, a distribution over t with $Z = 1$: this is logistic regression on the hidden units.

Multi-layer perceptrons stack soft perceptrons

The figure is the lecture's drawing of an MLP. The bottom row is the input layer, one unit $x_1, \dots, x_n$ per input, each drawn with a straight line inside because it passes its value on unchanged. The middle row is the hidden layer and the top row the output layer, with outputs $y_1, \dots, y_m$; their units carry the sigmoid inside. Every unit of a layer is connected to every unit of the layer below, and the small circles stand for units that are not drawn.

The inset in the corner is the reason for the hidden layer. It shows a plane of two inputs divided into two classes, C1 in blue and C2 in red, with a patch of C2 inside the region of C1 and an island of C1 inside that patch. One hyperplane divides the plane into two half-planes and cannot produce such regions; an MLP can.

Multi-layer perceptrons stack soft perceptrons

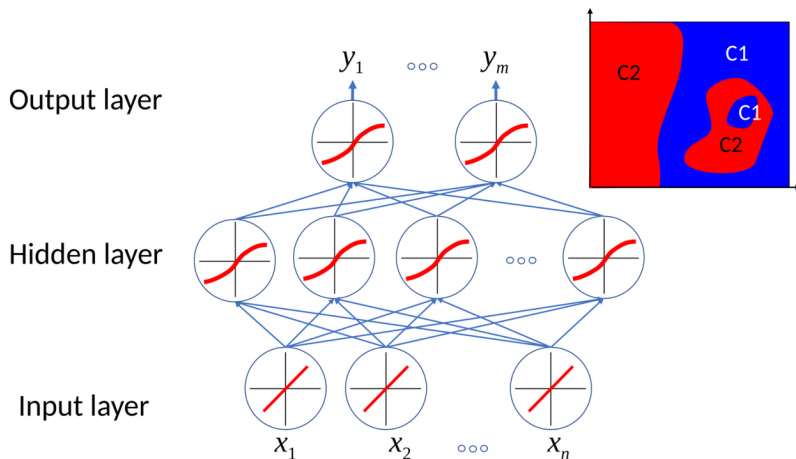


Figure 14: A multi-layer perceptron (from the lecture's animation of neural networks): an input layer, a hidden layer and an output layer, fully connected, and a decision region for two classes C1 and C2 that no single hyperplane produces.

Three hidden units combine three hyperplanes

The example network of the course notes can be trained on a problem that logistic regression cannot solve. The data are 120 points drawn uniformly in a disc of radius 1, the positive class, and 120 points in the ring between radii 1.55 and 2.45 around it, the negative class. The network has two inputs, three sigmoid hidden units and one sigmoid output, and its energy is the cross-entropy with $t = (y + 1)/2$. Gradient descent with momentum, which adds to each step 0.9 times the previous step, lowers the energy from 456.84 at the random start to 0.495, and the trained network classifies every point correctly. Logistic regression on the same data classifies 52.5% of the points correctly, about as well as guessing.

All the weights, $\vec{w}$ and $\{\vec{\Omega}^i\}$, are learnt together by gradient descent on the energy E . The training never says what each hidden unit should do; the division of labour between them emerges from the descent.

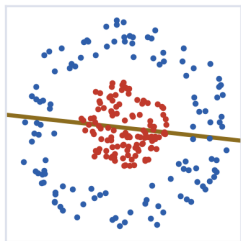
Three hidden units combine three hyperplanes

The trained network can be read off. Each hidden unit i is a soft perceptron with its own hyperplane $\vec{\Omega}^i \cdot \vec{x} + \Omega_0^i = 0$: z_i is close to 1 on one side and close to 0 on the other. The output unit takes a weighted sum of z_1, z_2, z_3 and passes it through a sigmoid. With suitable weights the sum exceeds the threshold only where each z_i is on the disc's side of its hyperplane: here z_1 and z_3 close to 1, and z_2 , whose output weight is negative, close to 0. So the positive class of the network is the intersection of three half-planes: a triangle, which here contains the disc and excludes the ring.

One hyperplane, the olive line of the first panel, can only cut the plane in two, and every cut leaves points of both classes on each side. More hidden units give polygons with more sides, closer to the circle.

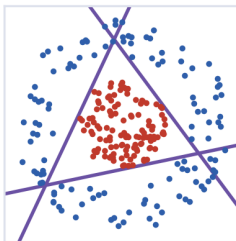
Three hidden units combine three hyperplanes

**logistic regression:
one hyperplane**



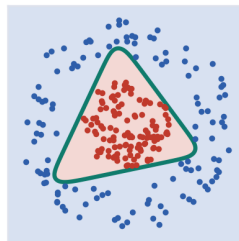
olive: cannot enclose

**the three
hidden units**



$$\vec{\Omega}^i \cdot \vec{x} + \Omega_0^i = 0$$

**the MLP:
inside all three**



teal: output 1/2

Figure 15: The disc (red dots, $y = +1$) and the ring (blue dots, $y = -1$). First panel: the hyperplane of logistic regression (olive). Second panel: the three hyperplanes of the trained hidden units (purple). Third panel: the output of the MLP; the teal curve is where the output is 1/2, and the shading marks the region that the network assigns to each class.

A deep network is a composition of differentiable functions

Deep networks, with many layers of hidden units, had their first large success in vision on object classification. Their output can be expressed as a composition of elementary functions, each depending on its own weights. For a network with two layers of hidden variables, an input image I , hidden layers H_1 and H_2 and an output O ,

$$H_1 = H_1(W_1, I), \quad H_2 = H_2(W_2, H_1), \quad O = O(W_3, H_2).$$

A special case is

$$H_1 = \text{ReLU}(W_1 I), \quad H_2 = \text{ReLU}(W_2 H_1), \quad O = \text{ReLU}(W_3 H_2),$$

where $\text{ReLU}(u) = \max(0, u)$ is the rectified linear unit, applied to each component. Here I is the vector of pixel values, in the role of x , H_1 and H_2 are vectors of hidden activities, the weights W_1, W_2, W_3 are matrices, and $W_1 I$ is a matrix times a vector: every hidden unit takes its own weighted sum of all the pixels. A bias can be included by appending a constant 1 to each vector.

A deep network is a composition of differentiable functions

The functions are composed to give the output $O = O(W, I)$, with all the weights collected in $W = (W_1, W_2, W_3)$. They are only required to be differentiable. ReLU has a corner at 0, where its derivative is set to 0 in practice; the choice rarely matters, because a total input of exactly 0 is rare.

The loss function compares the output with the ground truth T , the correct answer given by the training data: $L(O(W, I), T)$. When the network outputs a distribution, the loss is $-\log P(T | I, W)$, and W are the weights ω of the regression model $P(y | x, \omega)$. Learning the maximum likelihood estimate requires the partial derivatives of the loss with respect to every weight.

A deep network is a composition of differentiable functions

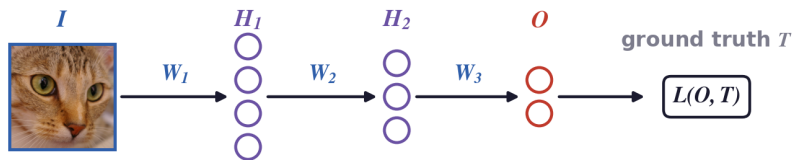


Figure 16: A deep network as a chain of functions. The input image I (blue frame) passes through the weights W_1 to the hidden layer H_1 , through W_2 to the hidden layer H_2 (purple circles), and through W_3 to the output O (red circles). The loss $L(O, T)$ compares the output with the ground truth T .

Backpropagation computes derivatives by the chain rule

The derivatives can be computed efficiently by exploiting the chain rule of differentiation. The method is known as the backpropagation algorithm, because errors are propagated backwards down the hierarchy (Rumelhart, Hinton and Williams, 1986). For W_3 the derivative is simple, because W_3 directly affects the output O :

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial O} \frac{\partial O}{\partial W_3}.$$

For W_2 , first compute how the loss depends on H_2 , then on W_2 :

$$\frac{\partial L}{\partial H_2} = \frac{\partial L}{\partial O} \frac{\partial O}{\partial H_2}, \quad \frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial H_2} \frac{\partial H_2}{\partial W_2}.$$

For W_1 , one step further back:

$$\frac{\partial L}{\partial H_1} = \frac{\partial L}{\partial H_2} \frac{\partial H_2}{\partial H_1}, \quad \frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial H_1} \frac{\partial H_1}{\partial W_1}.$$

Each derivative with respect to a layer is reused for the layer below it.

Backpropagation in the example network

The recipe can be written out for the MLP of the disc, with the energy of one example $E = -t \log y - (1 - t) \log(1 - y)$, with the label $t \in \{0, 1\}$ and the output $y = \sigma(u)$ of the network, where $u = \vec{w} \cdot \vec{z} + w_0$ is the total input of the output unit. The sigmoid has the derivative $\sigma'(u) = \sigma(u)(1 - \sigma(u))$, which gives $\partial E / \partial u = y - t$: the error at the output. Then

$$\frac{\partial E}{\partial w_i} = (y - t) z_i, \quad \frac{\partial E}{\partial w_0} = y - t.$$

The error reaches hidden unit i through the weight w_i , and its sigmoid multiplies it by its slope. Writing $u_i = \vec{\Omega}^i \cdot \vec{x} + \Omega_0^i$,

$$\delta_i = \frac{\partial E}{\partial u_i} = (y - t) w_i z_i (1 - z_i), \quad \frac{\partial E}{\partial \vec{\Omega}^i} = \delta_i \vec{x}, \quad \frac{\partial E}{\partial \Omega_0^i} = \delta_i.$$

Every derivative is a product of numbers already computed in the forward pass. Computed this way, the derivatives of the example network agree with finite differences, $\{E(\theta + \epsilon) - E(\theta - \epsilon)\} / (2\epsilon)$ for each weight θ , to better than 10^{-5} .

Backpropagation computes derivatives by the chain rule

The derivatives are products of vectors and matrices. $\partial L / \partial O$ is a vector with one entry per output unit, and $\partial O / \partial H_2$ is a matrix with one entry for each pair of an output unit and a hidden unit. The backward pass multiplies such matrices, from the output down, and costs about as much as one or two forward passes. Finite differences would need two forward passes for every single weight.

Backpropagation solves the credit assignment problem: deciding how much each weight deep inside the network contributed to an error at the output. The error goes back to give credit to weights that do not directly affect the output. The same procedure applies to networks with any number of layers, because computing the derivatives exploits the compositionality of the function $O = O(W, I)$.

Backpropagation computes derivatives by the chain rule

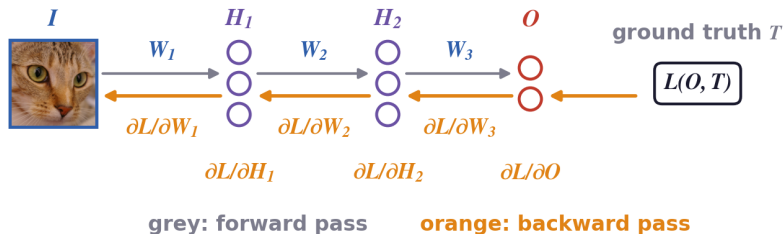


Figure 17: The forward and the backward pass. Grey arrows carry the forward pass from the image I through W_1 , W_2 and W_3 to the loss. Orange arrows carry the backward pass: $\partial L / \partial O$ from the loss, then $\partial L / \partial H_2$ and $\partial L / \partial H_1$ (the lower row of orange labels); from each of them follows the derivative for the weights of that layer (the upper row).

Stochastic gradient descent learns from random subsets

To train the network on a dataset $\{(I_n, T_n) : n = 1, \dots, N\}$, where the ground truth T_n plays the role of the label y , the derivatives of the loss are needed. Three update rules use them differently. Gradient descent on all the data, called steepest descent (SD) or batch mode, uses all N examples at each step:

$$W^{t+1} = W^t - \zeta_t \frac{1}{N} \sum_{n=1}^N \frac{d}{dW} L(O(W, I_n), T_n).$$

Stochastic gradient descent (SGD), or online learning, selects one example $n(t)$ at random at time step t and follows its gradient alone:

$$W^{t+1} = W^t - \zeta_t \frac{d}{dW} L(O(W, I_{n(t)}), T_{n(t)}).$$

Stochastic gradient descent learns from random subsets

In practice a random subset $N(t)$ of the data, a minibatch, is selected at each step:

$$W^{t+1} = W^t - \zeta_t \frac{1}{|N(t)|} \sum_{n \in N(t)} \frac{d}{dW} L(O(W, I_n), T_n).$$

Here $N(t)$ is a set of indices, and $|N(t)|$ the number of examples in it.

In the second wave of neural networks, in the 1980s, batch processing meant using all the training examples at the same time, and online learning meant stochastic gradient descent. In the third wave, deep learning, datasets are so big that it is impractical to use the full batch. Instead, subsets are selected at random, which is stochastic gradient descent with minibatches.

A random subset gives an unbiased estimate of the gradient

Why can one example stand in for all of them? If $n(t)$ is drawn uniformly from $1, \dots, N$, the expected value of its gradient is

$$\mathbb{E} \left[\frac{d}{dW} L(O(W, I_{n(t)}), T_{n(t)}) \right] = \frac{1}{N} \sum_{n=1}^N \frac{d}{dW} L(O(W, I_n), T_n),$$

the gradient of steepest descent. The same holds for the average over a minibatch drawn at random. Each SGD step is therefore right on average and wrong in detail: it is the true gradient plus noise.

The noise shrinks as the minibatch grows: its variance falls roughly as $1/|N(t)|$. The cost of a step grows as $|N(t)|$. For a fixed budget of computation, many cheap and noisy steps often lower the energy faster than a few exact ones, as long as the learning rate is small enough for the noise to average out over successive steps.

Stochastic gradient descent learns from random subsets

The figure compares the three rules on the disc example, from the same starting weights and with the same number of examples processed: 400 passes through the 240 examples. Each rule has its own learning rate: 2.0 for all N examples, 0.2 for one example and 0.6 for ten. After 400 passes the energy per example is 0.0755 for steepest descent, 0.0101 for minibatches of ten and 0.002 for single examples. Steepest descent took 400 steps, SGD with minibatches of ten 9,600 and SGD with one example 96,000.

Measured by passes through the data, which is the cost in computation, the rules that take more and noisier steps reach the lower energies. Per pass steepest descent updates the weights once, while SGD with single examples updates them 240 times.

Stochastic gradient descent learns from random subsets

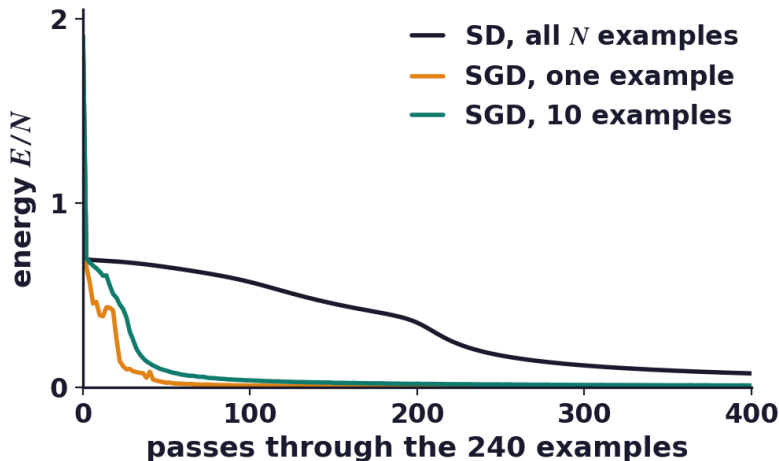


Figure 18: Energy per example of the disc MLP: steepest descent (black), SGD with single examples (orange), SGD with minibatches of ten (teal).

The energy is a non-convex function of the weights

Solving for the weights is a non-convex optimisation problem. This can be seen from the hidden symmetries of the network: there are several different ways to get the same input-output function. Permute the hidden units, and move their output weights with them: unit 1 becomes unit 3, unit 2 becomes unit 1, and so on. The network computes exactly the same function, so for any minimum of the energy there exist other minima with exactly the same value. Three hidden units can be permuted in $3! = 6$ ways.

The sigmoid adds a second symmetry. Since $\sigma(-u) = 1 - \sigma(u)$, reversing the signs of $\vec{\Omega}^i$ and Ω_0^i turns z_i into $1 - z_i$, and replacing w_i by $-w_i$ and w_0 by $w_0 + w_i$ leaves the output unchanged. With both symmetries every minimum of the example comes as a family of $2^3 \times 3! = 48$ weight vectors with the same energy, so the lowest energy is reached at 48 different points. A convex energy would keep that lowest value along the straight segment between any two of them; unless the minima are connected by such a valley of equal energy, there is no single global minimum.

The energy is a non-convex function of the weights

A convex function lies below each of its chords. In the figure, weights A are the trained network and weights B the same network with its hidden units permuted; both have the energy 0.495. On the straight line between them the energy rises to 969.66 at the midpoint, far above both ends, so E is not convex. At the midpoint each hidden unit has the average of two different sets of weights, and the network no longer separates the classes.

The symmetry means there is a set of minima that are equally good as solutions, and we only need to converge to one of them. Steepest descent on the energy risks getting stuck in a local minimum, lower than its surroundings and higher than the best, or at a saddle point, where the gradient vanishes and the energy rises in some directions and falls in others. The randomness of SGD helps it to escape some local minima, which are unlikely to lie at the same positions for all the training data. Results known as Robbins-Monro theory (Robbins and Monro, 1951) guarantee convergence when the learning rate falls off suitably, $\sum_t \zeta_t = \infty$ and $\sum_t \zeta_t^2 < \infty$, but only for problems that satisfy further conditions, such as a convex energy, and MLPs do not satisfy them.

The energy is a non-convex function of the weights

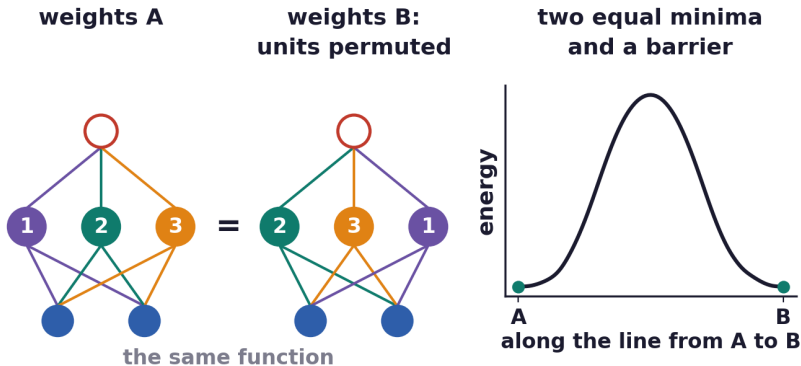


Figure 19: Weights A: the trained disc network, its three hidden units numbered and coloured. Weights B: the same units in a different order, each with its own input and output weights; the two networks compute the same function. The plot is the energy along the straight line from A to B: equal at both ends, a high barrier in between.

Each hidden neuron applies the same localized, linear filter

An MLP connects every hidden unit to every input. For images this is wasteful. A convolutional neural network (CNN) makes two restrictions. Each hidden neuron is local: it sees only a small window of the image. And the weights are shared: all the neurons of one map use the same weights W , applied at every position (here W is one filter). The weights form a filter, and the map of its responses is a feature map:

$$h_{ij} = \text{ReLU} \left(\sum_{u,v,c} W_{uvc} l_{i+u,j+v,c} + w_0 \right).$$

Here h_{ij} is the neuron of the map at position (i,j) , $l_{i+u,j+v,c}$ is the value of the image at row $i+u$, column $j+v$ and colour channel c , the offsets u,v run over the window of the filter (11×11 in AlexNet's first layer) and c over the three colours, red, green and blue, and w_0 is the bias of the filter. Every neuron of the map computes the same weighted sum, each over its own window.

Each hidden neuron applies the same localized, linear filter

The filter of the figure is a real one: filter 89 of the 96 filters that AlexNet learnt for its first layer, of size $11 \times 11 \times 3$ and shown enlarged. It is dark on the left and light on the right, so it responds most where the image goes from dark on the left to light on the right: it is a detector of vertical edges of one polarity.

The filter is applied at every position of the 224×224 photograph, followed by ReLU, which gives a map of 214×214 neurons, since $224 - 11 + 1 = 214$. Two windows are marked, in red and in orange, with the two neurons that they drive; both use the same 363 weights. AlexNet itself moves the window 4 pixels at a time, a stride of 4, which gives a coarser map; the figure uses a stride of 1, which is the formula as written.

Because the weights are shared, shifting the image shifts the feature map by the same amount and changes nothing else: the map is equivariant to shifts. An edge is detected wherever it appears.

Strictly, this operation is a correlation: a convolution in the mathematical sense would flip the filter. Deep learning calls it convolution all the same, and since the filter is learnt, the flip makes no difference.

Each hidden neuron applies the same localized, linear filter

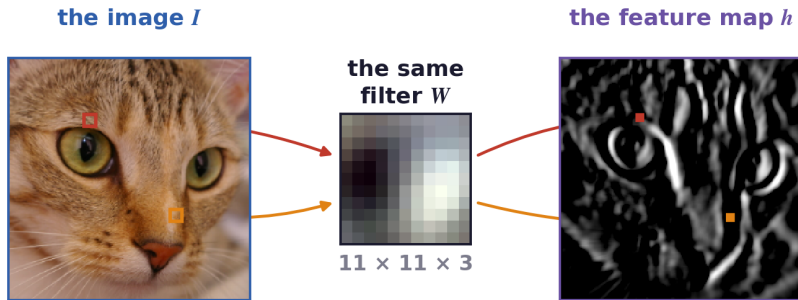


Figure 20: One filter applied everywhere. The photograph I (blue frame) with two 11×11 windows (red and orange squares); the filter W , the same for both windows; and the feature map h (purple frame), light where the response is large, with the two neurons that the windows drive (red and orange dots).

A bank of filters gives one feature map per filter

One filter detects one feature. A layer uses a bank of filters, each with its own weights, and each gives its own feature map. AlexNet's first layer has 96 filters of size $11 \times 11 \times 3$, so its output is 96 feature maps, one per filter.

Each map shows where its feature is. The six filters of the figure, also from AlexNet's first layer, look like a vertical edge, a horizontal edge, an oblique edge, an edge between two colours, a green blob and a pink blob, and their maps light up in different places. The map of the green-blob filter lights up only at the eyes, the only green parts of the photograph. The map of the horizontal filter lights up on the stripes of the fur.

A bank of filters gives one feature map per filter

The next layer treats the stack of maps as its input. Its filters are local in space and span many maps: a filter of AlexNet's second layer covers 5×5 positions of 48 of the first layer's maps (48 rather than 96, because the network was split between two graphics processors). Each of its neurons therefore combines the features found at nearby positions, for example an edge next to a blob.

In the figure every map is scaled so that its 99.5th percentile is white, because the maps differ widely in their largest values.

A bank of filters gives one feature map per filter

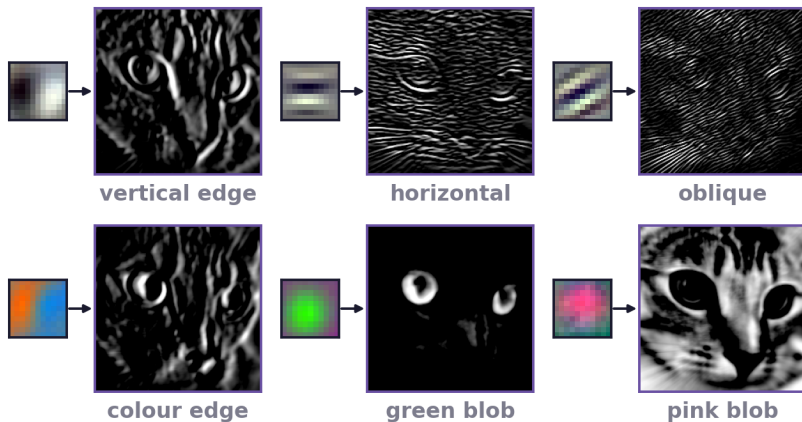


Figure 21: Six of the 96 first-layer filters of AlexNet (small squares, enlarged), each applied at every position of the photograph and followed by ReLU. The grey maps are the responses, light where large; the labels describe what each filter looks like.

Local pooling keeps the largest response in each window

Pooling reduces the resolution of a map. Max pooling keeps the largest value of each window:

$$p_{ij} = \max_{0 \leq u, v \leq 2} h_{2i+u, 2j+v}.$$

In AlexNet the windows are 3×3 and 2 pixels apart, so neighbouring windows overlap by one row or column. A map of 54×54 becomes one of 26×26 , since $(54 - 3)/2 + 1 = 26$ when rounded down.

The lower resolution has two effects. Deeper filters then cover a larger part of the image with the same number of weights: a 3×3 filter applied to the pooled map spans the width of seven positions of the map before pooling. And a small shift of the image changes the pooled map less than the feature map, because a maximum over a window moves only when the largest value leaves the window.

Local pooling keeps the largest response in each window

The figure measures the second effect. The feature map is the first-layer map of filter 89 in AlexNet, of size 54×54 with the stride of 4. The photograph is then shifted 4 pixels to the right, one step of the stride, and the maps are compared position by position, without undoing the shift. The correlation between the maps of the shifted and the unshifted image, a number that is 1 for maps that agree perfectly, is 0.386 before pooling and 0.735 after it. Small shifts are tolerated.

AlexNet pools after a local response normalisation, which divides each response by a quantity computed from the responses of neighbouring maps at the same position. For the figure, max pooling with the network's own window and stride is applied directly to the rectified map. Applied to the normalised map instead, the same pooling gives exactly the output of the network's pooling layer.

Local pooling keeps the largest response in each window

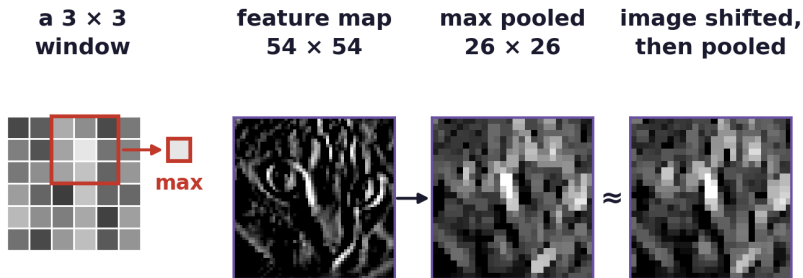


Figure 22: Max pooling. The schematic shows a 3×3 window (red square) on a map and the single value that it keeps. The three maps are the feature map of filter 89 at stride 4, the same map max pooled with 3×3 windows 2 apart, and the pooled map of the photograph shifted 4 pixels; the two pooled maps are similar (correlation 0.735).

A deep CNN computes a hierarchy of feature maps

A deep CNN repeats the pattern. Each layer applies local filters to the maps of the layer below, and pooling reduces the resolution from time to time. The deeper maps are coarser, and their units see larger regions of the image: a unit of AlexNet's fifth convolutional layer, conv5, sees 163×163 of the 224×224 pixels, most of the cat's face.

The hierarchy is motivated by conjectures about the structure of the human and mammalian visual system, where neurons in successive areas respond to larger regions and to more complex patterns. Object classification is a coarse task, and a hierarchy suits it: it can learn features that are invariant to irrelevant details of the object, such as its exact position. The fully connected layers at the top output $P(y | x)$ over the 1,000 classes of ImageNet.

The receptive field grows with depth

The region of the image that a unit sees is its receptive field. It follows from two numbers per layer: the size k of its window and its stride s . Let r be the receptive field of a unit of the layer, in pixels, and d the distance in pixels between neighbouring units. For the image itself $r = 1$ and $d = 1$, and each layer updates them by

$$r \mapsto r + (k - 1) d, \quad d \mapsto d s.$$

For AlexNet: conv1 ($k = 11$, $s = 4$) gives $r = 11$, $d = 4$; pool1 ($k = 3$, $s = 2$) gives $r = 19$, $d = 8$; conv2 ($k = 5$) gives $r = 51$; pool2 gives $r = 67$, $d = 16$; and conv3, conv4 and conv5 ($k = 3$ each) give $r = 99$, 131 and 163.

The sizes of the maps follow from the same numbers: a window of size k moved in steps of s over n positions fits $\lfloor (n - k)/s \rfloor + 1$ times. For conv1 on a 224×224 image this is 54. The paper states 55, which needs an input of 227 pixels. A layer that pads its input with p rows and columns of zeros on each side fits $\lfloor (n + 2p - k)/s \rfloor + 1$ windows; conv2 to conv5 are padded so that they keep the size of their input.

A deep CNN computes a hierarchy of feature maps

The figure passes the photograph through AlexNet and shows four of the maps of three layers: conv1, the first convolutional layer, with 96 maps of 54×54 ; conv2 with 256 maps of 26×26 ; and conv5 with 256 maps of 12×12 . (With an input of 227 pixels the sizes would be the paper's 55, 27 and 13.) From each layer the four maps with the most varied activity (a large spread of values over many active units) are shown, each scaled so that its 99.5th percentile is white. The conv1 maps still show the cat. The conv2 maps show blobs of texture and the regions of the eyes. The conv5 maps are coarse patterns of a few active units, which the fully connected layers turn into the label Egyptian cat.

A deep CNN computes a hierarchy of feature maps

AlexNet: four maps each of conv1, conv2 and conv5

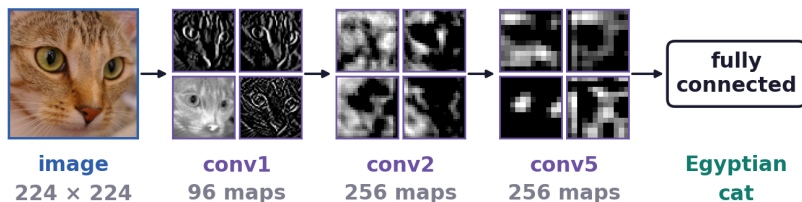


Figure 23: The photograph (blue frame) and four feature maps each of conv1, conv2 and conv5 of AlexNet (purple frames), light where the response is large. The fully connected layers output the label, here Egyptian cat.

A CNN is an MLP with local, shared weights

The three models of the lecture differ in their connections. A perceptron is one unit and one hyperplane. An MLP has fully connected layers: every unit sees the whole layer below. A CNN has local windows and shared weights.

The restriction matters for images. Fully connecting a 224×224 colour image, 150,528 inputs, to a layer of the size that the paper gives for AlexNet's first layer, $55 \times 55 \times 96 = 290,400$ units, would need $150,528 \times 290,400 \approx 44$ billion weights (with the 54 positions of a 224-pixel input, 279,936 units and 42 billion weights). AlexNet's first convolutional layer needs about 35,000: 96 filters of $11 \times 11 \times 3$, that is 34,848 weights and 96 biases, shared by all positions.

A CNN is a special case of an MLP: the weights are zero outside each window and equal across positions. The soft perceptron, the MLP and the CNN are all trained in the same way: by maximum likelihood, with the gradients computed by backpropagation and followed by SGD. The perceptron with a step threshold is the exception, and is trained by Rosenblatt's rule.

A CNN is an MLP with local, shared weights

Backpropagation needs one addition for shared weights. A shared weight affects the loss through every position where it is used, so its derivative is the sum of the derivatives at all positions:

$$\frac{\partial L}{\partial W_{uvc}} = \sum_{i,j} \frac{\partial L}{\partial h_{ij}} \frac{\partial h_{ij}}{\partial W_{uvc}}.$$

Sharing lets the filter learn from every position of every image at once. With far fewer weights, a CNN also needs fewer training examples than a fully connected network would to fit its weights.

A CNN is an MLP with local, shared weights

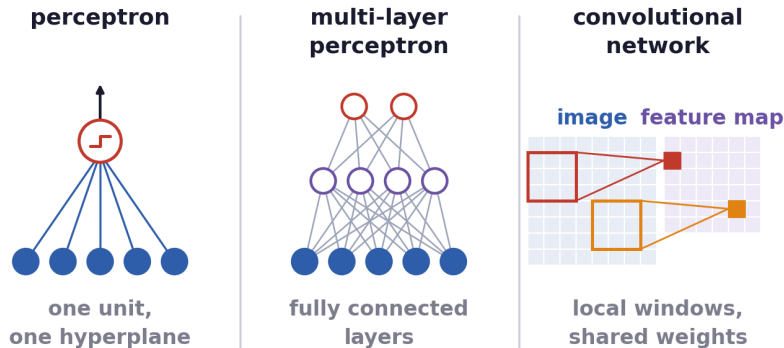


Figure 24: Three networks. Perceptron: five inputs (blue) and one unit with a step threshold (red). Multi-layer perceptron: every unit is connected to every unit of the layer below (grey lines), with a hidden layer in purple. Convolutional network: two windows of the image (red and orange squares) drive two units of the feature map through the same weights.

AlexNet classifies ImageNet images into 1,000 classes

ImageNet (Deng et al., 2009) is a database of millions of photographs, each labelled with the name of the object it shows. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) uses a subset of 1,000 classes, with about 1.2 million training images, 50,000 validation images and 150,000 test images. In 2012 a deep convolutional network trained on it, known as AlexNet after its first author, won the challenge by a wide margin (Krizhevsky, Sutskever and Hinton, 2012).

Hinton's slides on the network sum it up in three points. It is deep: 7 hidden “weight” layers, plus the output layer. It is entirely supervised: every training image comes with its label, and more data is better. And it is learned: all the feature extractors are initialised at white Gaussian noise, independent random numbers, and learned from the data. The edge and blob detectors of the first layer were never designed by hand.

AlexNet classifies ImageNet images into 1,000 classes

The figure shows validation images (test images in the paper's Figure 4), which the network did not see during training, with its answers. Under each image is its correct label in bold type, and under that the five labels that the network rates most probable, most probable first. The length of the bar behind a label is the probability that the network assigns to it, and the bar is red when the label is the correct one.

For the four images of the figure, a mite on a white surface, a container ship, a motor scooter at night and a leopard, the correct label is the network's first choice. For the ship almost all the probability goes to it; the bar of the mite is short but well ahead of black widow. The alternatives are sensible: black widow and tick for the mite, lifeboat and fireboat for the ship, jaguar and cheetah for the leopard.

AlexNet classifies ImageNet images into 1,000 classes



Figure 25: Four ILSVRC validation images, their correct labels and the network's five most probable labels; bar length is probability, a red bar marks the correct label (figure: G. Hinton, slides on Krizhevsky, Sutskever and Hinton, 2012).

The top-five error counts a correct label among five guesses

ILSVRC scores a classifier in two ways. The top-1 error is the fraction of test images whose correct label is not the network's first choice. The top-5 error is the fraction whose correct label is not among its five most probable labels. The second measure forgives an image that holds several objects, or a label that ImageNet splits into near-synonyms. AlexNet reached top-1 and top-5 error rates of 37.5% and 17.0% on the ILSVRC-2010 test set, and a variant won ILSVRC-2012 with a top-5 error of 15.3%, against 26.2% for the second-best entry.

The four images of the figure are harder. For the grille, the front of a car, the first choice is convertible and grille comes second. For the mushroom, the first choice is agaric, a kind of mushroom. For the cherry the dalmatian behind the cherries wins, and cherry is missing from the top five. For the Madagascar cat, a ring-tailed lemur, the network lists four monkeys and the indri, another lemur. All four are top-1 errors, cherry and Madagascar cat are also top-5 errors, and all are mistakes a person could understand.

The top-five error counts a correct label among five guesses

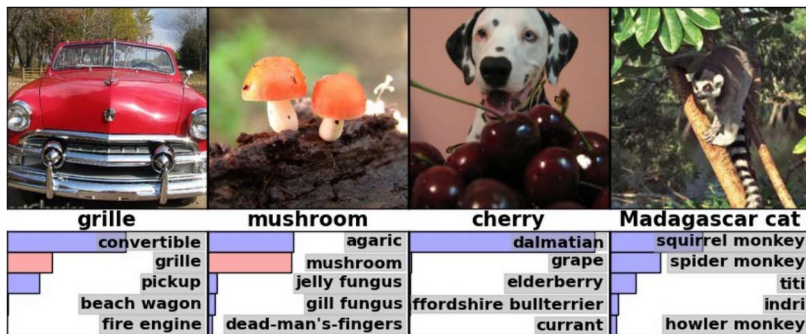


Figure 26: Four more validation images, drawn in the same way as the mite, the ship, the scooter and the leopard. Grille and mushroom have a red bar in second place; cherry and Madagascar cat have no red bar, so the correct label is outside the top five (figure: G. Hinton, slides on Krizhevsky, Sutskever and Hinton, 2012).

AlexNet: five convolutional and three fully connected layers

AlexNet has five convolutional layers followed by three fully connected layers. The first convolutional layer, conv1, has 96 filters of $11 \times 11 \times 3$ applied with a stride of 4: its window moves 4 pixels at a time. The next layers use windows of 5×5 (conv2) and 3×3 (conv3, conv4 and conv5). Max pooling follows conv1, conv2 and conv5. The fully connected layers fc6 and fc7 have 4,096 units each, and fc8 has 1,000 output units, one per class, followed by the softmax.

The network has 650,000 neurons, 60 million parameters and 630 million connections. The activities of fc7, a 4096-dimensional vector, are the final feature layer: the description of the image from which the class is read off. The network was trained with SGD on two graphics processors (GPUs) for about a week, five to six days in the paper. Each layer was split between the two GPUs, which communicated only at certain layers; the drawing merges the two halves into one stream with the full numbers of maps.

AlexNet: five convolutional and three fully connected layers

The 60 million parameters can be counted from the sizes of the layers. The five convolutional layers hold about 2.3 million weights: 34,848 in conv1, 307,200 in conv2, 884,736 in conv3, 663,552 in conv4 and 442,368 in conv5, where conv2, conv4 and conv5 see only the maps on their own GPU. The fully connected layers hold the rest: fc6 connects the $6 \times 6 \times 256 = 9,216$ values of the last pooled maps to 4,096 units, 37.7 million weights; fc7 has $4,096^2 = 16.8$ million and fc8 4.1 million. With the biases the total is 60,965,224.

So the convolutional layers, which do most of the computation, hold less than 4% of the parameters, and the first fully connected layer holds more than half. The sizes in the drawing are the paper's: the paper states an input of 224 pixels, but its 55 positions for conv1 need 227, and with 224 pixels conv1 gives 54. In the computed network the last pooling layer pads the 12×12 maps of conv5 with one row and one column of zeros, so fc6 still receives maps of 6×6 .

AlexNet: five convolutional and three fully connected layers

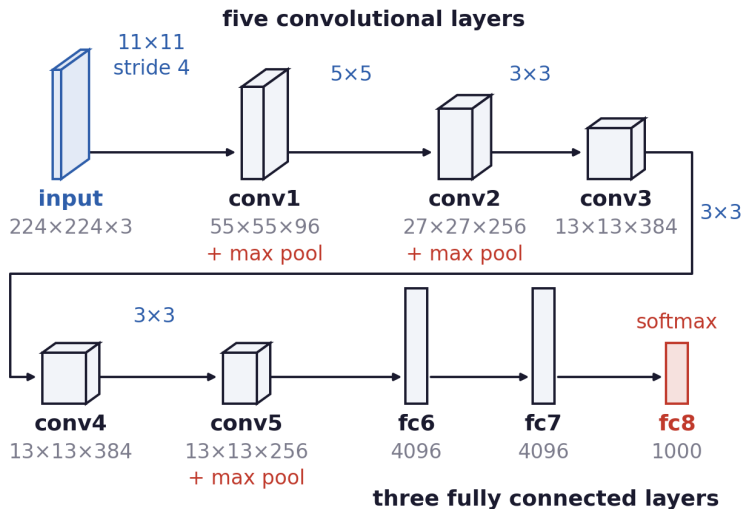


Figure 27: AlexNet's layers, redrawn after Figure 2 of Krizhevsky, Sutskever and Hinton (2012) as in Hinton's slides. Each box is a stack of feature maps labelled with its size; blue labels give filter sizes, red labels mark max pooling, tall bars are fully connected layers.

Training maximises the log-probability of the correct label

The network has one output unit per class. Write x_i for the total input to output unit i , the weighted sum that the unit receives from fc7. The softmax turns the 1,000 total inputs into probabilities:

$$f(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{1000} \exp(x_j)}.$$

Every $f(x_i)$ is positive and they add up to one, and $f(x_i)$ is the network's probability $P(y = i | I, \omega)$ that the image I shows class i . In this notation x_i is Hinton's total input, not the image.

Training maximises the log-probability of the correct label t , $\log f(x_t)$, summed over the training images. This is maximum likelihood for the regression model $P(y | I, \omega)$, with a network as the family of distributions. It is carried out by stochastic gradient descent and the backpropagation algorithm, which is just repeated application of the chain rule.

The gradient of the log softmax is target minus prediction

Backpropagation starts at the output, with the derivative of $\log f(x_t) = x_t - \log \sum_j \exp(x_j)$ with respect to each total input:

$$\frac{\partial \log f(x_t)}{\partial x_i} = [i = t] - f(x_i),$$

where $[i = t]$ is 1 if $i = t$ and 0 otherwise. The total input of the correct class is pushed up by $1 - f(x_t)$, and that of every other class is pushed down by its probability. A confident and correct network, with $f(x_t)$ close to 1, receives almost no correction.

With two classes the softmax is logistic regression:

$$f(x_1) = \frac{e^{x_1}}{e^{x_1} + e^{x_2}} = \frac{1}{1 + e^{-(x_1 - x_2)}} = \sigma(x_1 - x_2),$$

a sigmoid of the difference of the two total inputs. The softmax is the generalisation of logistic regression to many classes.

Training maximises the log-probability of the correct label

The paper gives the details of the stochastic gradient descent. Each step uses a minibatch of 128 images, with momentum 0.9 and weight decay 0.0005, a small pull of every weight towards zero. The learning rate starts at 0.01 and is divided by 10, three times in all, when the error on the validation set stops improving. Training took roughly 90 passes through the 1.2 million images.

The figure is Hinton's summary of training. In each column a circle is a convolutional layer, a square a fully connected hidden layer, and the thin bar at the top the third fully connected layer, fc8, with its 1,000 output units. In the forward pass the image goes up through the layers to the softmax. In the backward pass the gradient of $\log f(x_t)$ comes down through the same layers, and every weight is adjusted.

Training maximises the log-probability of the correct label

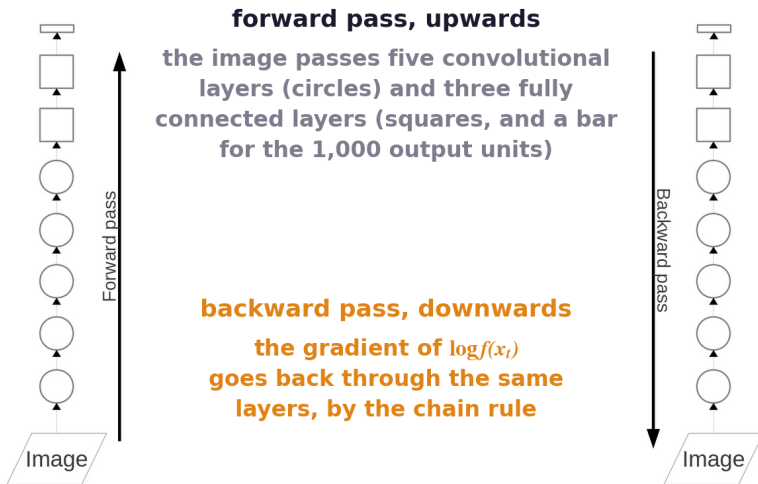


Figure 28: The forward pass, upwards, and the backward pass, downwards, through the five convolutional layers (circles) and the three fully connected layers (two squares and the output bar) of AlexNet (columns from G. Hinton's slides).

Rectified linear units train much faster than tanh units

Every unit computes a total input and passes it through an output function. In Hinton's notation a unit with three inputs receives

$$x = w_1 f(z_1) + w_2 f(z_2) + w_3 f(z_3),$$

the weighted sum of the outputs $f(z_k)$ of the units below, where z_k is the total input of unit k , and its own output is $f(x)$. Here f is the output function of one unit, different from the softmax f of the output layer.

Earlier networks used $f(x) = \tanh(x)$, an S-shaped function from -1 to 1 , a rescaled sigmoid. AlexNet uses the rectified linear unit, $f(x) = \max(0, x)$. Hinton's verdicts: tanh is very bad, slow to train; the rectified linear unit is very good, quick to train.

The reason is the slope $f'(x)$. Backpropagation multiplies the error by the slope of every unit it passes through. For tanh the slope is $1 - \tanh^2 x$, which is at most 1 and close to 0 once $|x|$ exceeds about 2 : the unit saturates, and the error that passes through it is nearly erased. Over many layers the products of small slopes make the gradients of the early layers vanish.

Rectified linear units train much faster than tanh units

The rectified linear unit has slope 1 wherever it is active, $x > 0$, and passes the error back unchanged; where it is inactive, $x < 0$, it passes nothing, and the unit is simply off for that image. Its output is also cheap to compute.

The paper measures the difference. On CIFAR-10, a dataset of the Canadian Institute for Advanced Research (CIFAR) with 60,000 small colour images of 32×32 pixels in 10 classes (Krizhevsky, 2009), a four-layer convolutional network with rectified linear units reaches a training error of 25% six times faster than the same network with tanh units. Deep convolutional networks with rectified linear units train several times faster than their equivalents with tanh units.

Rectified linear units train much faster than tanh units

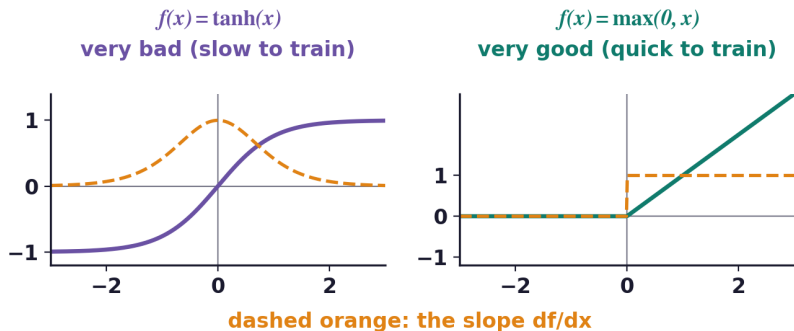


Figure 29: The two output functions (solid) and their slopes df/dx (dashed orange). The slope of tanh is 1 at $x = 0$ and falls towards 0 on both sides; the slope of the rectified linear unit is 0 for $x < 0$ and 1 for $x > 0$.

Data augmentation and dropout reduce overfitting

AlexNet has 60 million real-valued parameters and 650,000 neurons, and it overfits a lot: with so many parameters it can fit the 1.2 million training images closely and still make many errors on new images. Two remedies are used.

The first is data augmentation: more training images are made from the existing ones by changes that keep the label. Every training image is rescaled so that its shorter side has 256 pixels, and its central 256×256 square is kept; the network is trained on random 224×224 patches of it and on their horizontal reflections. There are 33×33 positions for a patch and two orientations, 2,178 versions of each image (the paper quotes a factor of 2,048), although the patches of one image are far from independent. The paper also varies the intensities of the colour channels. At test time the network averages its predictions at five 224×224 patches, the four corners and the centre, and at their reflections: ten patches in all.

Data augmentation and dropout reduce overfitting

a 256×256 image



224×224 patches



**right
column:
reflected**

Figure 30: Data augmentation on the photograph. The 256×256 image with four random 224×224 patches outlined in four colours, and the four patches themselves; the patches of the right column are reflected left to right.

Dropout trains many thinned networks that share their weights

The second remedy is dropout. During training, each hidden unit of the two fully connected hidden layers, fc6 and fc7, is set to zero with probability 0.5, independently for every training image. A unit that is turned off contributes nothing to the forward pass and receives no error in the backward pass. Each image therefore trains a different thinned network, a random half of the units, and all these networks share their weights.

A unit can then never rely on the presence of particular other units, and has to learn features that are useful in many combinations. At test time all the units are used and their outputs are multiplied by 0.5, so that the total input of the next layer has the size it had on average during training. This is a fast approximation to averaging the predictions of the exponentially many thinned networks. Dropout roughly doubles the number of iterations that training needs to converge.

Dropout trains many thinned networks that share their weights

grey crosses: hidden units turned off for this image

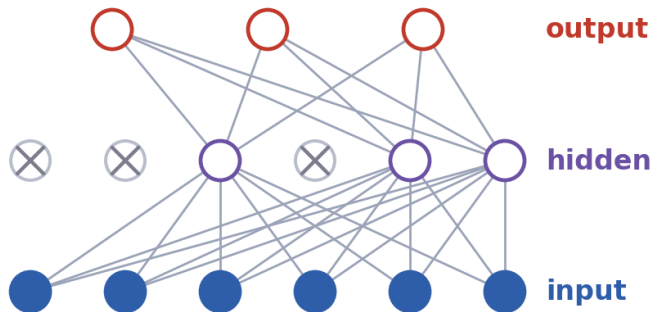


Figure 31: Dropout for one training image. Input units (blue), one hidden layer (purple) and output units (red); the grey crosses are the hidden units turned off for this image, and only the connections of the units that remain are drawn.

The first layer learns oriented edges and colour blobs

The figure shows the 96 kernels, the filters of $11 \times 11 \times 3$, that AlexNet learned for its first layer. The top 48 were learned on one GPU and the rest on the other. The network has learned a variety of kernels selective for orientation and for spatial frequency, the spacing of stripes: edges and gratings at many orientations and spacings, as well as various coloured blobs. The kernels on the first GPU are largely colour-agnostic and those on the second largely colour-specific, and the paper reports that this division occurs in every run, whatever the random initialisation.

Many of the kernels resemble the receptive fields of simple cells in the primary visual cortex (V1), the first cortical area to receive visual input. A simple cell responds best to an edge or a bar of a particular orientation at a particular position (Hubel and Wiesel, 1962). The network was given only images and labels, and learned detectors of the same kind.

The first layer learns oriented edges and colour blobs



Figure 32: The 96 first-layer kernels of AlexNet, each shown as a small colour image; the upper three rows were learned on one GPU and the lower three on the other (figure: G. Hinton, slides on Krizhevsky, Sutskever and Hinton, 2012).

The 4096-dimensional features retrieve similar images

The activities of the last hidden layer, the 4,096 units of fc7, form a vector that describes the image. Two images whose vectors are close in Euclidean distance are similar in the eyes of the network. The retrieval experiment uses this: for each query image from the ILSVRC-2010 test set, the training images whose vectors are nearest are retrieved.

In the figure the first column holds the query images, and the rest of each row the retrieved training images. The retrieved images have final feature vectors close to the query's, although their pose differs: the elephants, for example, are seen from the side and from the front. They are often far from the query in pixel space, where images are compared pixel by pixel and a change of pose makes two images of the same object very different. The last hidden layer has learned a description of the image that ignores such changes.

The 4096-dimensional features retrieve similar images

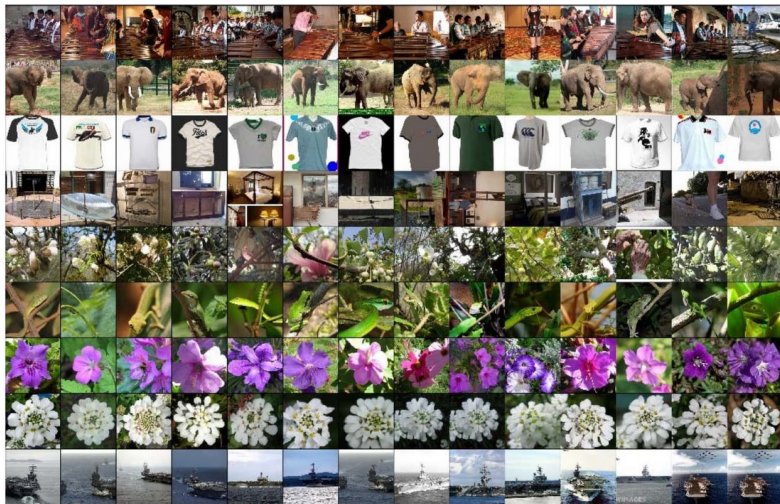


Figure 33: Retrieval with the 4096-dimensional features. The first column holds query images from the ILSVRC-2010 test set; the other columns hold the training images with the nearest feature vectors (figure: G. Hinton, slides on Krizhevsky, Sutskever and Hinton, 2012).

Fine-detail tasks need a decision at every pixel

Deep networks had their first large success in vision on object classification (Krizhevsky et al., 2012). This is a coarse task: the aim is to classify an image as a single object, one label for the whole image. It suits a hierarchy that pools the image down to a few coarse maps and learns features invariant to the details.

Deep networks were rapidly adapted to fine-detail tasks, such as estimating the edges in an image or semantic segmentation, which assigns every pixel the class of the object it belongs to. Both tasks require results per pixel, hence fine detail, and the pooling that helps classification throws away exactly the positions they need. A fully convolutional network (FCN) (Long, Shelhamer and Darrell, 2015) replaces the fully connected layers by convolutions, so that the network outputs a coarse map of class scores in place of a single vector, and then upsamples the map to the resolution of the image, adding detail from earlier layers.

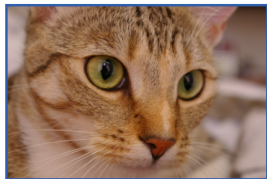
Fine-detail tasks need a decision at every pixel

The figure contrasts the two tasks. For classification, AlexNet gives the photograph of the cat one label, Egyptian cat. For segmentation, an FCN labels every pixel: here an FCN built on ResNet-50, a residual network of 50 layers (residual networks add skip connections, which let very deep networks train). It was trained on images of the Common Objects in Context (COCO) dataset labelled with the 20 object classes of the PASCAL Visual Object Classes (VOC) benchmark plus a class for background.

On the photograph of the cat the FCN labels 78% of the pixels as cat and the rest as background, and the boundary it draws follows the outline of the animal. On a photograph of an astronaut it labels 43% of the pixels as person and nearly all the rest as background. The output is a picture of the scene, with the objects in their places, where classification gives a word.

Fine-detail tasks need a decision at every pixel

**classification:
one label**

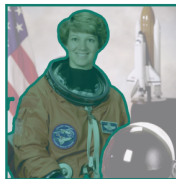


**AlexNet:
“Egyptian cat”**

**segmentation: a label at
every pixel, by an FCN**



**teal: cat
grey: background**



**teal: person
grey: background**

Figure 34: Classification and segmentation. First panel: the photograph and AlexNet’s label. Second and third panels: the segmentation by an FCN, with the pixels of the object class in teal and those of the background in grey; the teal curve is the boundary between them. The photographs “chelsea” and “astronaut” (NASA) are from the scikit-image sample data, in the public domain.

Edges occur at different scales in the image

Edge detection shows why a per-pixel task needs several levels of a network. Edges occur at different scales in the image. Intuitively, edges at different scales can be detected by convolving the image with a Gaussian of the corresponding scale and taking the derivative. The scale is the standard deviation σ of the Gaussian G_σ , and the strength of the edges at scale σ is the magnitude of the gradient of the blurred image,

$$|\nabla(G_\sigma * I)|,$$

where $*$ is convolution and ∇ takes the derivatives along the rows and the columns. Blurring with G_σ averages the image over a region of radius about σ .

Edges occur at different scales in the image

Fine edges, where the intensity changes and changes back within a few pixels, as in fur, whiskers and texture, are averaged out by Gaussians with large variances, and so they give cues for edges only at the smallest scales. Texture edges are of this kind, and a detector of object boundaries does not want them. Large edges remain as the image is blurred with larger Gaussians. There is information about edges at all scales, as exploited by statistical edge detectors, such as the Bayesian edge detector of Lecture 4, which combines filters of several sizes.

Edges occur at different scales in the image

The figure applies the recipe to the grey version of the photograph, with $\sigma = 1, 2, 4$ and 8 pixels. At $\sigma = 1$ the map is full of the fur, the whiskers and the stripes. As σ grows the texture fades, and at $\sigma = 8$ only the large structures remain: the outlines of the eyes, the ears and the head. Each map is scaled so that its 99.5th percentile is white; in absolute terms the gradients weaken with the blur, and the 99.5th percentile falls from 0.097 at $\sigma = 1$ to 0.017 at $\sigma = 8$, in units of the grey level per pixel, with grey levels from 0 to 1.

Edges occur at different scales in the image

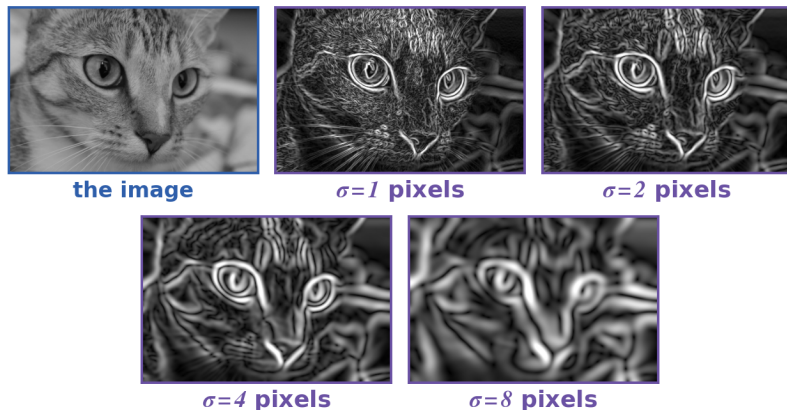


Figure 35: The grey photograph (blue frame) and the edge strength $|\nabla(G_\sigma * I)|$ at four scales (purple frames), light where the gradient is large; each map is scaled to its own 99.5th percentile.

HED gives the network a loss at every level

Holistically-nested edge detection (HED) (Xie and Tu, 2015) captures this intuition. It is built on VGG-16, a 16-layer CNN of the Visual Geometry Group (VGG) in Oxford (Simonyan and Zisserman, 2015) arranged in five stages; within a stage the maps keep their resolution, and pooling halves it between stages. After the last convolutional layer of each stage, HED attaches a side layer, which turns the maps of that stage into a map of edge probabilities, upsampled to the size of the image. These side layers have access to the ground truth and have a loss function of their own, so every level of the network is supervised directly. The side outputs are also combined into a fused output, which has its own loss. The idea of supervising the hidden layers of a network directly is called deep supervision (Lee et al., 2015).

The combination uses information from all scales. The lower layers, which have higher spatial resolution, give accurate estimates of the positions of edges; the higher layers validate that there is an edge. HED wants to detect boundary edges, which are large and have evidence at most scales of the network.

HED gives the network a loss at every level

The loss of side output m is a cross-entropy in which the two classes are weighted:

$$\ell_{\text{side}}^{(m)} = -\beta \sum_{j \in Y_+} \log P(y_j = 1 | I) - (1 - \beta) \sum_{j \in Y_-} \log P(y_j = 0 | I).$$

Here j runs over pixels, Y_+ is the set of edge pixels and Y_- the set of non-edge pixels of the ground truth, Y is the set of all pixels, $\beta = |Y_-|/|Y|$, and $P(y_j = 1 | I)$ is the side output at pixel j . The total loss is

$$\sum_{m=1}^5 \alpha_m \ell_{\text{side}}^{(m)} + \mathcal{L}_{\text{fuse}},$$

where the α_m weight the side losses (a different α from the decision rule of Bayes decision theory) and $\mathcal{L}_{\text{fuse}}$ is the loss of the fused output.

HED gives the network a loss at every level

As in statistical edge detection, the loss must strongly penalise false negatives. About 90% of the pixels of a natural image are non-edges, so $\beta \approx 0.9$, and a missed edge pixel costs $\beta/(1 - \beta) \approx 9$ times as much as a false alarm, a non-edge pixel labelled edge.

The weighting removes an easy way out. With equal weights, a network that outputs $P(y_j = 1 | I) = 0.1$ at every pixel, the fraction of edges, has a loss per pixel of $-0.1 \log 0.1 - 0.9 \log 0.9 \approx 0.33$ and finds no edge at all. With the weights, a constant output p has a loss per pixel of $0.09(-\log p - \log(1 - p))$, smallest at $p = 1/2$, an output that says nothing; to lower the loss further the network must find the edges.

HED gives the network a loss at every level

VGG-16: the resolution halves at each stage

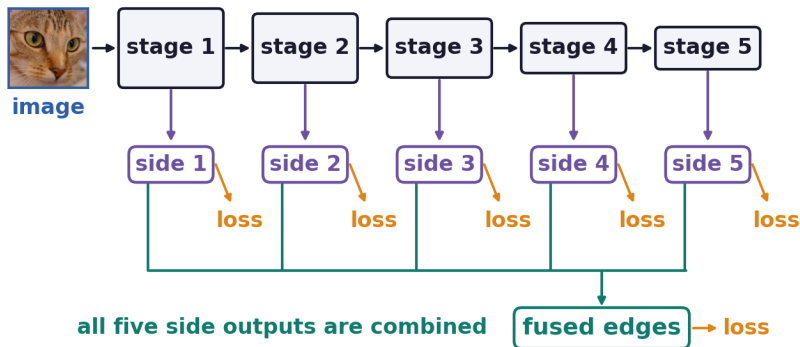


Figure 36: HED. The image passes the five stages of VGG-16, whose resolution halves from stage to stage. Each stage has a side output (purple) with its own loss (orange). The teal lines collect the five side outputs into the fused edge map, which has a loss of its own.

Residual networks add a skip connection to each block

The course notes stop at HED: they cover supervision at different levels, and skip connections are the other important idea of modern architectures. Deeper networks should be able to do at least as well as shallower ones, since extra layers could simply copy their input. In practice deeper plain networks, stacks of layers with no skips, can train worse: He et al. (2016) found that on the CIFAR-10 dataset a plain network of 56 layers had a higher training error, and also a higher test error, than one of 20 layers. This is a failure of optimisation, since the training error itself is higher, and it is called degradation.

A residual network adds a skip connection around each block of layers:

$$H_{l+1} = H_l + F(H_l, W_l).$$

In the original network a ReLU follows the addition; the form written here, with a pure identity, is that of the later paper on identity mappings. The block computes a residual F , which is added to its input H_l . If copying the input is the best a block can do, it only has to learn $F = 0$, which is easy. Residual networks of up to 152 layers won ILSVRC 2015.

Residual networks add a skip connection to each block

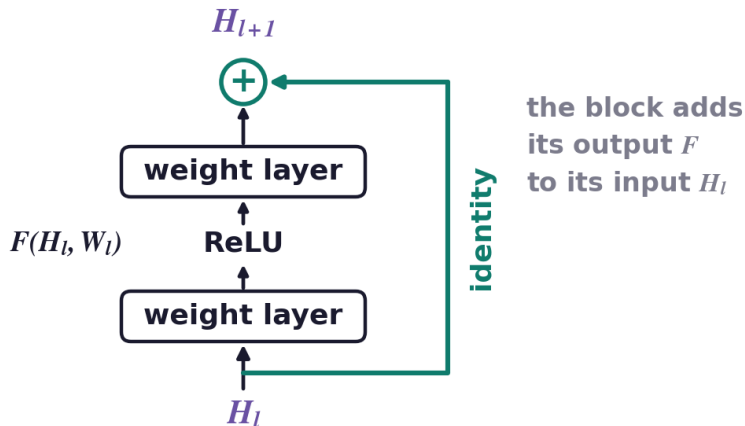


Figure 37: A residual block. The input H_l passes two weight layers with a ReLU between them, which compute $F(H_l, W_l)$; the teal skip connection carries H_l unchanged around them, and the circle adds the two to give H_{l+1} .

The error reaches every block through the identity path

The skip connection also changes the backward pass. By the chain rule applied to $H_{l+1} = H_l + F(H_l, W_l)$,

$$\frac{\partial L}{\partial H_l} = \frac{\partial L}{\partial H_{l+1}} \left(1 + \frac{\partial F}{\partial H_l} \right),$$

where for vectors the 1 is the identity matrix. Adding the blocks from l to a deeper layer m gives $H_m = H_l + \sum_{k=l}^{m-1} F(H_k, W_k)$, and so (He et al., 2016, on identity mappings)

$$\frac{\partial L}{\partial H_l} = \frac{\partial L}{\partial H_m} \left(1 + \frac{\partial}{\partial H_l} \sum_{k=l}^{m-1} F(H_k, W_k) \right).$$

The error reaches every block through the identity path

The first term carries the error from layer m straight back to layer l , multiplied by no weight matrix and no slope. In a plain network, $H_{l+1} = F(H_l, W_l)$, the error reaches layer l only through a product of one factor $\partial F / \partial H_k$ per layer, which can shrink towards zero or grow without bound over many layers: forty factors of 0.9 multiply to 0.015, and forty factors of 1.1 to 45. Each block learns only the residual F ; the skip connection carries H forward, and the error back, unchanged.

Forty residual blocks train where forty plain layers stall

The figure repeats the comparison on a small problem: classifying the points of two interleaved spirals, 600 points in the plane. The plain networks have 3 or 40 hidden layers of 16 units with ReLU after an input layer; the residual network has 40 blocks, each with two weight layers as in the drawing of the block, so it is the deepest of the three. Every network ends in a logistic output unit. It is trained by SGD with momentum 0.9, a learning rate of 0.01 and minibatches of 64. The training energy, the average cross-entropy, is plotted every 25 iterations.

A plain network of 3 layers falls from 0.69 to 0.045 in 3,000 iterations. A plain network of 40 layers stays near $0.69 \approx \log 2$, the energy of a network that outputs $1/2$ for every point, apart from two brief dips, and ends at 0.696. A residual network of 40 blocks falls fastest, to 0.033, along a noisier curve.

Forty residual blocks train where forty plain layers stall

The toy shows the effect found by He et al.: the deeper plain network trains worse. Its cause here is different. The toy networks have no batch normalisation, which rescales the inputs of each layer to zero mean and unit variance over a minibatch (Ioffe and Szegedy, 2015), and the signal of the plain 40-layer network collapses in the forward pass, its output becomes constant, and the gradients of the early layers vanish. He et al. used batch normalisation and argue that their degradation is unlikely to be caused by vanishing gradients.

Forty residual blocks train where forty plain layers stall

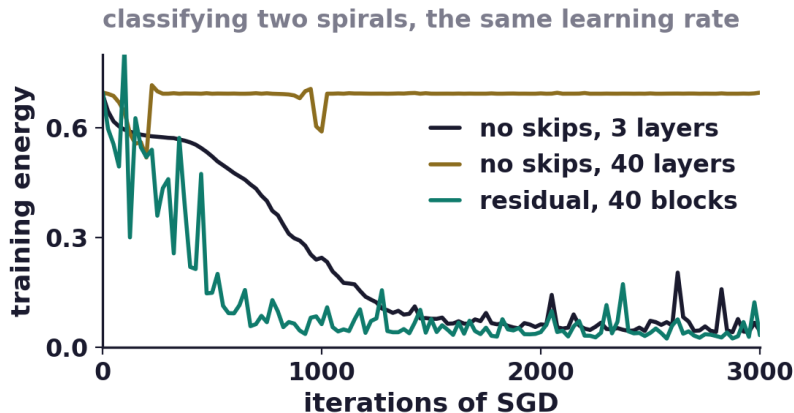


Figure 38: Training energy against iterations of SGD on two spirals, for a plain network of 3 layers (black), a plain network of 40 layers (olive) and a residual network of 40 blocks (teal), all with the same learning rate.

U-Net: skip connections carry fine detail forward

The U-Net (Ronneberger, Fischer and Brox, 2015) uses skip connections for per-pixel tasks. Its architecture consists of a contracting path to capture context and a symmetric expanding path that enables precise localisation. In the contracting path each level applies two 3×3 convolutions with ReLU and then a 2×2 max pooling, which halves the resolution, and the number of feature maps doubles from level to level: 64, 128, 256, 512 and 1,024 at the bottom. In the expanding path each level upsamples the maps and applies a 2×2 convolution, an up-convolution that halves the number of maps, concatenates the result with the correspondingly cropped maps of the contracting path (cropped because convolutions without padding leave the contracting maps larger), and applies two 3×3 convolutions. A final 1×1 convolution gives a score for every class at every pixel.

The skip connections copy the maps of each contracting level and concatenate them into the expanding path. The bottom of the U knows what is in the image, since its units see large regions, and it has lost where exactly; the copied maps still have the precise positions. The expanding path combines the two.

U-Net: skip connections carry fine detail forward

The U-Net was designed for biomedical image segmentation, where few annotated images are available, and was trained with heavy data augmentation. It is now used far more widely. Many diffusion models, which generate images by removing noise step by step, use a U-Net to predict the noise (Ho, Jain and Abbeel, 2020). The expanding path is often called the decoder, and the contracting path the encoder.

In the figure the numbers are the numbers of feature maps at each level. The boxes go down the contracting path, the descending arm of the U, in blue, and up the expanding path, the ascending arm, in purple; the red arrows are max pooling, the orange arrows upsampling, and the teal arrows the skip connections across the U.

U-Net: skip connections carry fine detail forward

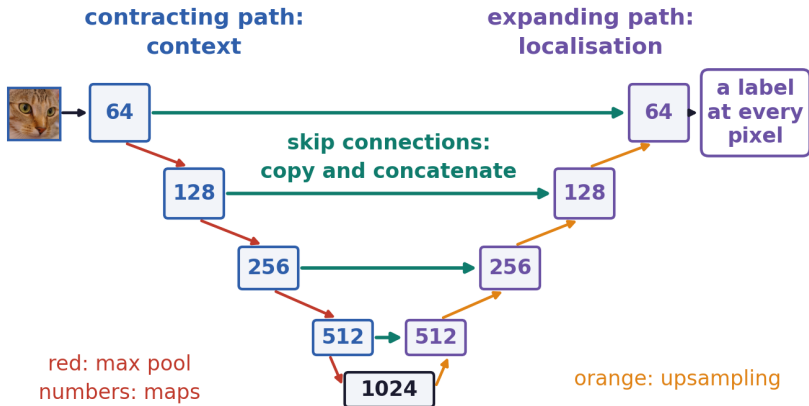


Figure 39: The U-Net. Blue boxes: the contracting path; purple boxes: the expanding path; the numbers are feature maps. Red arrows: max pooling; orange arrows: upsampling; teal arrows: the skip connections, which copy and concatenate. The output is a label at every pixel.

Summary

- ▶ Decisions need only the posterior; regression learns $P(y | x, \omega)$ by maximum likelihood.
- ▶ $P(y | x, \omega)$ may approximate the posterior well for some tasks; datasets are too small to check.
- ▶ Linear regression is least squares; logistic regression softens the perceptron's hyperplane.
- ▶ An MLP stacks soft perceptrons; SGD follows the gradients that backpropagation computes.

Summary

- ▶ A CNN is an MLP with local, shared filters and pooling: a hierarchy of feature maps.
- ▶ AlexNet: eight layers, ReLU, augmentation and dropout; the first layer learns edges and blobs.
- ▶ Per-pixel tasks need fine detail: HED supervises every level with its own loss.
- ▶ Skip connections let ResNet train very deep networks and give U-Net its fine detail.

References

- [1] A. L. Yuille. Regression and neural networks; Bayes decision theory and regression. Lecture notes, Johns Hopkins University, 2024.
- [2] C. F. Gauss. *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*. Perthes and Besser, Hamburg, 1809.
- [3] A.-M. Legendre. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot, Paris, 1805.
- [4] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [5] A. B. J. Novikoff. On convergence proofs on perceptrons. In *Symposium on the Mathematical Theory of Automata*, 12:615–622, Polytechnic Institute of Brooklyn, 1962.
- [6] Y. Freund and R. E. Schapire. Large margin classification using the perceptron algorithm. *Machine Learning*, 37(3):277–296, 1999.

References

- [7] D. E. Rumelhart, G. E. Hinton and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [8] H. Robbins and S. Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei. ImageNet: a large-scale hierarchical image database. *CVPR*, 248–255, 2009.
- [10] A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet classification with deep convolutional neural networks. *NIPS* 25, 1097–1105, 2012.
- [11] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [12] D. H. Hubel and T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *Journal of Physiology*, 160(1):106–154, 1962.

References

- [13] J. Long, E. Shelhamer and T. Darrell. Fully convolutional networks for semantic segmentation. *CVPR*, 3431–3440, 2015.
- [14] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *ICLR*, 2015.
- [15] S. Xie and Z. Tu. Holistically-nested edge detection. *ICCV*, 1395–1403, 2015.
- [16] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang and Z. Tu. Deeply-supervised nets. *AISTATS*, 562–570, 2015.
- [17] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. *CVPR*, 770–778, 2016.
- [18] K. He, X. Zhang, S. Ren and J. Sun. Identity mappings in deep residual networks. *ECCV*, 630–645, 2016.

References

- [19] S. Ioffe and C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. *ICML*, 448–456, 2015.
- [20] O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. *MICCAI*, LNCS 9351, 234–241, 2015.
- [21] J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS* 33, 6840–6851, 2020.